

Django Complete Mastery Guide — Finance Tracker Edition

Your situation: You built this Finance Tracker 6 months ago, learned basics from Hitesh Choudhary, and need to re-learn everything from scratch to advanced — concepts, commands, flow, and how to build/modify by yourself.

How to use this guide: Every concept follows the same structure: **What is it** → **Why** → **How it works internally** → **When loaded** → **What breaks** → **Beginner mistake** → **Your project example**

Code examples use YOUR actual Finance Tracker project files.

TABLE OF CONTENTS

- [Chapter 0 — Your Project Architecture Map](#)
 - [Chapter 1 — Mental Model](#)
 - [Chapter 2 — Project Structure \(Every File Explained\)](#)
 - [Chapter 3 — All Django Commands](#)
 - [Chapter 4 — ORM Mastery](#)
 - [Chapter 5 — Templates Deep Dive](#)
 - [Chapter 6 — Authentication & Authorization](#)
 - [Chapter 7 — Middleware](#)
 - [Chapter 8 — Forms Deep Dive](#)
 - [Chapter 9 — DRF Bridge](#)
 - [Chapter 10 — Development Workflows & Recipes](#)
 - [Chapter 11 — Deployment & Production](#)
 - [Chapter 12 — Interview Preparation](#)
 - [Chapter 13 — Troubleshooting Cheatsheet](#)
-

CHAPTER 0 — YOUR PROJECT ARCHITECTURE MAP

0.1 What is this project?

A **Finance Tracker** web application built with Django 5.x and PostgreSQL. Users can:

- Register/login (email + Google OAuth via django-allauth)
- Track income, expenses, and investments with multi-currency support
- Set monthly budgets per category with overrun alerts (email via Resend SDK)
- View dashboards, monthly reports, anomaly detection, AI insights (Gemini/OpenAI)
- Import bank statements (CSV/PDF)
- Upload receipt files

0.2 Your 3 Django Apps — What Each Does

```

finance_tracker/      ← PROJECT (settings, root urls, wsgi/asgi)
|
├── core/              ← APP 1: User management
|   ├── models.py      → UserProfile (extends User with preferred_currency)
|   ├── views.py       → register, login, logout, profile
|   ├── forms.py       → UserRegistrationForm, UserUpdateForm, UserProfileForm
|   └── urls.py        → /auth/register/, /auth/login/, /auth/logout/,
/auth/profile/
|   ├── signals.py     → post_save auto-creates UserProfile when User is
created
|   └── apps.py        → CoreConfig with ready() importing signals
|
├── finance/           ← APP 2: Core financial data (the main app)
|   ├── models.py      → Currency, Category, Transaction, Budget, Notification
|   ├── views.py       → CRUD for transactions, categories, budgets,
notifications, import
|   ├── forms.py       → TransactionForm, CategoryForm, BudgetForm,
TransactionFilterForm
|   ├── services.py    → Business logic: create/update/delete transactions,
budget checks
|   ├── context_processors.py → Injects unread_notification_count into every
template
|   ├── currency_utils.py → Multi-currency conversion (source → USD → target)
|   ├── import_service.py → CSV/PDF bank statement parsing and import
|   ├── admin.py       → Admin UI for all 5 models
|   └── urls.py        → /transactions/, /categories/, /budgets/,
/notifications/, /import/
|
├── reports/           ← APP 3: Analytics and reporting
|   ├── views.py       → dashboard, monthly_report, anomaly, ai_insights
|   ├── selectors.py   → All aggregation queries (totals, breakdowns, trends)
|   ├── anomaly.py     → Spending anomaly detection
|   ├── ai_insights.py → Gemini/OpenAI powered financial advice
|   └── urls.py        → /dashboard/, /reports/, /anomalies/, /ai-insights/
|
├── templates/         ← All HTML templates (project-level)
|   ├── base.html      → Master layout (nav, messages, CSS)
|   ├── home.html      → Landing page
|   ├── includes/      → Reusable partials (nav_notifications.html)
|   ├── core/          → login.html, register.html, profile.html
|   ├── finance/       → All transaction/category/budget/notification templates
|   └── reports/        → Dashboard, monthly report, anomaly, AI templates
|
├── static/css/        ← Developer-owned CSS
├── manage.py          ← CLI entry point
├── requirements.txt    ← Python dependencies
├── Dockerfile         ← Container build
├── docker-compose.yml  ← Local dev with PostgreSQL
├── render.yaml        ← Render.com deployment config
└── .env              ← Environment variables (never committed)

```

0.3 How a Request Flows Through YOUR Project

Example: User visits `/transactions/` (logged in)

1. Browser sends GET `/transactions/`
↓
2. Django Middleware Chain (top → down):
 SecurityMiddleware → WhiteNoiseMiddleware → SessionMiddleware
 → CommonMiddleware → CsrfViewMiddleware → AuthenticationMiddleware
 → MessageMiddleware → XFrameOptionsMiddleware → AccountMiddleware
 ↓
3. URL Resolver:
`finance_tracker/urls.py` → `path('', include('finance.urls'))`
`finance/urls.py` → `path('transactions/', views.transaction_list,`
`name='transaction_list')`
 ↓
4. View: `finance/views.py` → `transaction_list(request)`
 - Calls service: `services.get_user_transactions(request.user, filters)`
 ↓
5. Service Layer: `finance/services.py` → `get_user_transactions()`
 - Builds QuerySet:
`Transaction.objects.filter(user=user).select_related('category', 'currency')`
 - Applies filters (type, category, date_from, date_to)
 ↓
6. ORM → SQL:
`SELECT t.*, c.*, cur.* FROM finance_transaction t`
`JOIN finance_category c ON t.category_id = c.id`
`JOIN finance_currency cur ON t.currency_id = cur.id`
`WHERE t.user_id = %s ORDER BY t.date DESC, t.created_at DESC`
 ↓
7. Template Engine:
`render(request, 'finance/transaction_list.html', context)`
 - Context processors inject: request, user, messages, unread_notification_count
 - Template extends base.html → nav, messages block, content block
 ↓
8. HttpResponse → Middleware (bottom → up) → Browser renders HTML

0.4 How settings.py Wires Everything Together

YOUR `finance_tracker/settings.py` connects all components:

```
# Apps registered – controls model discovery, admin, templates, static finders
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'django.contrib.sites',          # Required by allauth
```

```

    'allauth',                                # Social auth
    'allauth.account',
    'allauth.socialaccount',
    'allauth.socialaccount.providers.google',
    'core',                                    # YOUR app 1
    'finance',                                 # YOUR app 2
    'reports',                                # YOUR app 3
]

# Middleware chain – order matters!
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'whitenoise.middleware.WhiteNoiseMiddleware',          # Static files in prod
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'allauth.account.middleware.AccountMiddleware',          # allauth
]

# URL routing root
ROOT_URLCONF = 'finance_tracker.urls'

# Template engine configuration
TEMPLATES = [{
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
    'DIRS': [BASE_DIR / 'templates'],          # Project-level templates
    'APP_DIRS': True,                          # Also look in app/templates/
    'OPTIONS': {
        'context_processors': [
            'django.template.context_processors.debug',
            'django.template.context_processors.request',
            'django.contrib.auth.context_processors.auth',
            'django.contrib.messages.context_processors.messages',
            'finance.context_processors.notification_count', # YOUR custom
processor
        ],
    },
}]

# Database – PostgreSQL via dj-database-url
DATABASES = {'default': dj_database_url.config(default=config('DATABASE_URL'))}

# Static & Media
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
STATIC_ROOT = BASE_DIR / 'staticfiles'
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

```

CHAPTER 1 — MENTAL MODEL

1.A What literally happens when a user types a URL and hits Enter (end-to-end)

What is it, literally?

A chain of handoffs from the browser to DNS → TCP/TLS → HTTP request → server process → Django internals → response returned.

Minimal ASCII flow:

```
Browser (http request)
-> DNS resolver (domain -> IP)
-> TCP handshake (client <-> server)
-> TLS handshake (if https)
-> HTTP request bytes sent
-> Web server (Nginx/Dev server)
-> WSGI/ASGI server (Gunicorn/Uvicorn)
-> Django process
    -> Middleware (request phase)
    -> URL resolver (urls.py)
    -> View (views.py)
    -> ORM/Services (models.py, queries)
    -> Template render or JSON serialization
    -> HttpResponse object
    -> Middleware (response phase)
-> WSGI/ASGI -> Web server -> Browser (response)
```

Why does Django need this explained concretely?

So you can map where to debug latency, 404s, auth issues, template errors or DB problems. Interviews often ask "what happens when you type example.com" — this map is the canonical answer.

How does it work internally? (hand-offs, what runs)

- **Browser resolves domain:** local resolver → recursive resolver → authoritative DNS; returns IP.
- **Browser opens TCP 3-way handshake** (SYN, SYN-ACK, ACK) to the IP:port (80/443).
- **If HTTPS, TLS handshake** negotiates symmetric keys (ClientHello -> ServerHello -> certificates -> keys).
- **HTTP request bytes** are sent (headers + body) to server process.
- **On production:** Nginx (reverse proxy) accepts connection, optionally terminates TLS, proxies to upstream worker over HTTP or Unix socket. **On dev:** Django development server (runserver) receives it.
- **WSGI** (sync) or **ASGI** (async) server receives request and calls Django application callable.
- **Django bootstrap for request:**
 - settings and app registry were loaded at startup (see runserver sequence below).
 - request enters middleware chain: each middleware.request(request) hook may run; it can short-circuit and return a response.

- URL resolver inspects urlpatterns (root -> includes) to find first matching pattern; captures path params.
- Resolver loads target view and calls it (function or view callable from Class Based View `as_view()` wrapper). At this point `request.user` was attached by AuthenticationMiddleware earlier.
- View executes: validating inputs (forms/serializers), calling service layer, performing ORM queries.
- ORM translates QuerySet operations to SQL, sends to DB via DB driver (psycopg2 for Postgres), receives rows, produces model instances.
- If view returns template response, Django renders template by loading template file, compiling nodes, evaluating with context, escaping content, producing HTML string.
- View returns HttpResponse (status, headers, body). Response-phase middleware runs in reverse order; it can modify headers, cookies, streaming, or do logging.
- WSGI/ASGI server receives response and writes bytes out. Reverse-proxy returns to client. Browser parses and renders.

When does each handoff happen? (startup vs per-request)

- **DNS/TCP/TLS:** per-request from client side (outside Django).
- **WSGI/ASGI server and Django import/app registry initialization:** at process start (startup).
- **Middleware, URL resolution, views, ORM, template rendering:** per-request.
- **Connection pooling (DB):** pools are created at startup and used per-request.

What breaks if missing or wrong?

- DNS misconfiguration → client can't resolve domain (browser shows DNS error).
- TLS/cert misconfigured → browser shows insecure site or fails handshake.
- Nginx misproxy settings → 502 Bad Gateway.
- Missing URL pattern → 404.
- Error in view template → `TemplateDoesNotExist` or `TemplateSyntaxError`.
- ORM mismatch (migrations not applied) → `FieldDoesNotExist` / column missing or DB errors.
- Middleware order wrong → authentication/session not available (e.g., `AuthenticationMiddleware` must come after `SessionMiddleware`).

A common beginner mistake

Assuming Django is responsible for DNS/TLS — those are infra. Another frequent error: putting database-heavy work in template (blocking request and causing timeouts).

1.B What happens when you run `python manage.py runserver`

What is it, literally?

A development server process that boots Django for local development with auto-reload. Example minimal invocation:

```
python manage.py runserver 0.0.0.0:8000
```

Why does Django need this command?

It provides a quick local HTTP server that loads your Django app and helps you iterate (auto-reload on code change).

How does it work internally?

- `manage.py` sets `DJANGO_SETTINGS_MODULE` and calls Django management command runner.
- `runserver` command:
 - Bootstraps Django: sets up settings, logging, app registry (loads `INSTALLED_APPS` and `AppConfig.ready()`).
 - Loads URLconf (`ROOT_URLCONF`). Validates.
 - Initializes autoreload watcher: runs a reloader that monitors files and restarts workers on change.
 - Starts a single-process threaded HTTP server in dev (not suitable for production): for Python 3 it uses `django.utils.autoreload` + `socketserver` or uses ASGI server if configured to run async dev server.
 - Optionally prints server info and handles signals (Ctrl-C).
- On incoming requests the same request cycle runs (middleware → url resolver → view → response).

When does Django load/use components here?

- Settings and `INSTALLED_APPS` are loaded on the `runserver` startup.
- Migration state, admin models, signal handlers from apps are set up during app registry initialization at startup.
- Template loaders, staticfiles finders are set up at startup but used per-request or by `collectstatic`.

What breaks if missing/wrong?

- If `DJANGO_SETTINGS_MODULE` wrong → `ImportError` at startup.
- If settings reference missing env vars → crash on startup.
- If `INSTALLED_APPS` contains an app that raises on import → startup failure.
- If `ALLOWED_HOSTS` and `DEBUG=False` → `runserver` will reject host headers (in dev, `DEBUG=True` by default).

A common beginner mistake

Relying on `runserver` behavior for production performance/security (e.g., serving static files directly or ignoring using Gunicorn/Nginx).

1.C MVT (Model-View-Template) — concrete request example walked through each layer

What is it, literally?

- **Model:** Python class inheriting `models.Model` mapping to DB.
- **View:** function/class handling request.
- **Template:** DTL file rendering HTML.

Minimal concrete snippet using YOUR project:

models.py

```
from django.db import models
class Transaction(models.Model):
    user = models.ForeignKey("auth.User", on_delete=models.CASCADE)
    amount = models.DecimalField(max_digits=12, decimal_places=2)
    date = models.DateField()
    description = models.TextField(blank=True)
```

views.py

```
from django.shortcuts import render
def tx_list(request):
    qs = Transaction.objects.filter(user=request.user).order_by('-date')[:50]
    context = {"transactions": qs}
    return render(request, "finance/transaction_list.html", context)
```

templates/finance/transaction_list.html

```
{% extends "base.html" %}
{% block content %}
<ul>
{% for tx in transactions %}
    <li>{{ tx.date }} - {{ tx.amount }} - {{ tx.description }}</li>
{% endfor %}
</ul>
{% endblock %}
```

Why does Django need MVT specifically?

It provides separation: Models hold data, Views orchestrate logic/formatting, Templates present UI. This keeps concerns distinct and code testable.

How does it work internally? (execution order)

1. Request → URL matches `tx_list`.
2. View `tx_list` is called with `request`.
3. The view executes `Transaction.objects.filter(...)` — QuerySet lazy, when iterated/templates evaluated it triggers database `SELECT` with `WHERE user_id = ... ORDER BY date DESC LIMIT 50`.
4. QuerySet returns model instances; view builds context dict and calls template renderer.
5. Template engine compiles and renders HTML, escapes variables, returns `HttpResponse`.

When does Django load/use each piece?

- **Models** are imported at startup (app registry); queries happen per-request.
- **Template** compilation may be cached; actual rendering happens per-request unless cached.

- **Views** are callables invoked per-request.

What breaks if missing/wrong?

- If model field name changed but view still uses old name → `AttributeError` at template rendering or view.
- If template path wrong → `TemplateDoesNotExist` when view calls render.
- If view forgets to restrict `user=request.user` → **data leak**.

A common beginner mistake

Returning `QuerySet` to template without slicing or prefetching leads to N+1 issues in loops when referencing related objects.

1.D How project-level settings, apps, and the project relate (small diagram)

What is it, literally?

`settings.py` configures `INSTALLED_APPS`; each app is a Python package with models/views/templates; project wires apps together via `ROOT_URLCONF`.

ASCII diagram:

```
settings.py
INSTALLED_APPS = [
    "django.contrib.auth",
    "finance.apps.FinanceConfig",    <-- app registered
    "reports.apps.ReportsConfig",
]

project root (urls.py)
-> includes finance.urls, reports.urls

App (finance)
- models.py
- views.py
- urls.py
- templates/finance/*.html
```

Why does Django need this relationship?

Django's app registry uses settings to register models, admin, signals, migrations and template/static lookup.

How does it work internally?

At startup Django imports settings, then calls `django.setup()` which constructs `Apps` registry: it imports each `AppConfig` (from `apps.py`) and runs ready hooks. That populates the global model registry and admin.

When loaded?

At process startup (runserver/gunicorn worker start or management command invocation). Some lazy imports happen later.

What breaks if missing/wrong?

- Missing app from `INSTALLED_APPS` → migrations won't be considered, admin won't show models, templates/static under app not found by app-specific finders.
- Import error in `AppConfig.ready()` → startup failure.

Common beginner mistake

Editing `INSTALLED_APPS` after startup in code or using wrong dotted path for `AppConfig`.

PART 1 — CHECK YOURSELF (2–3 short questions + answers)

Q1 — Explain step-by-step what happens (inside Django) between URL resolution and template output for `tx_list` above.

A1 — URL resolver matches path → view called → QuerySet built → iteration triggers DB SELECT → ORM returns model instances → view creates context → `render()` finds template → template engine compiles + renders with context → returns `HttpResponse`.

Q2 — If your template throws `TemplateDoesNotExist` only on production, what are 3 likely causes?

A2 — (1) collectstatic/template files not deployed or `TEMPLATE_DIRS` misconfigured; (2) app missing in `INSTALLED_APPS`, so app template loader not found; (3) wrong path/filename differing by case on case-sensitive FS in prod.

Q3 — Why must `AuthenticationMiddleware` come after `SessionMiddleware` in `MIDDLEWARE`?

A3 — `AuthenticationMiddleware` relies on session data (`request.session`) to load `request.user`. If `SessionMiddleware` hasn't run, `request.session` will be missing and auth won't work.

CHAPTER 2 — PROJECT STRUCTURE (Every File Explained)

Every Django project has a standard set of files. For each file below: what it is literally, why Django needs it, how it works internally, when it's loaded, what breaks if wrong, and common beginner mistakes.

manage.py

What is it, literally?

A thin command-line entry-point script created at project bootstrap. Example (auto-generated):

```
#!/usr/bin/env python
import os
import sys

if __name__ == "__main__":
    os.environ.setdefault("DJANGO_SETTINGS_MODULE", "finance_tracker.settings")
    from django.core.management import execute_from_command_line
    execute_from_command_line(sys.argv)
```

Why does Django need this file?

It sets `DJANGO_SETTINGS_MODULE` for local commands and delegates to Django's management command dispatcher so you can run `runserver`, `migrate`, `shell`, etc., with project context.

How does it work internally?

It sets the environment variable, then imports Django's `execute_from_command_line` which bootstraps Django, loads settings, and runs the requested management command.

When is it used?

Only when you invoke `python manage.py <command>` — every development/management operation.

What breaks if it's missing/wrong?

You can still call `django-admin` but `manage.py` is the convenient per-project entry. If it points at wrong settings module or sets wrong env, commands will load incorrect config or fail to import settings.

Common beginner mistake

Hard-coding settings or paths in `manage.py` or committing a `manage.py` that points to local-only settings.

settings.py

What is it, literally?

A Python module with assignments that configure Django: `INSTALLED_APPS`, `MIDDLEWARE`, `DATABASES`, `TEMPLATES`, `STATIC_*`, `MEDIA_*`, `AUTH_USER_MODEL`, `SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS`, etc.

Minimal snippet:

```
BASE_DIR = Path(__file__).resolve().parent.parent
SECRET_KEY = config('SECRET_KEY')
DEBUG = config('DEBUG', default=True, cast=bool)
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'finance',
```

```
        'reports',
    ]
    DATABASES = {
        'default': dj_database_url.config(default=config('DATABASE_URL'))
    }
    STATIC_URL = '/static/'
    MEDIA_URL = '/media/'
```

Why does Django need it?

Central configuration that controls how the framework and your apps behave.

How does it work internally?

At startup, Django reads `DJANGO_SETTINGS_MODULE` and imports this module. Django core modules and apps read relevant settings during setup (app registry, middleware, DB connections, templates, staticfiles finders).

When is it loaded/used?

Loaded at process startup (runserver, gunicorn workers, management commands). Settings may be read lazily by some subsystems but initial import is at boot.

What breaks if missing/wrong?

- Missing `DJANGO_SETTINGS_MODULE` or import errors => startup failure.
- Wrong DB settings => DB connection errors.
- `DEBUG` left True in prod => sensitive info leakage.
- Missing `SECRET_KEY` => security warnings or startup failure.
- Misconfigured `STATIC_ROOT` => static serving broken in production.

Common beginner mistakes

- Committing `SECRET_KEY` to VCS.
- Having different settings in dev and prod without clear environment override pattern.
- Putting heavy logic in settings (avoid side effects).

urls.py (project-level)

What is it, literally?

Root URLConf mapping URL patterns to views or including app url modules. Example:

```
from django.urls import path, include
from django.contrib import admin

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('finance.urls', namespace='finance')),
```

```
    path('reports/', include('reports.urls', namespace='reports')),  
]
```

Why does Django need it?

The URL resolver traverses this list to locate the correct view for incoming requests.

How does it work internally?

Django builds a URL resolver tree from `urlpatterns` at startup. For each incoming request path it tries patterns in order; when `include()` is hit, it delegates resolution to the included module with the remaining path.

When is it loaded/used?

Loaded at startup (imported from `ROOT_URLCONF`). Resolution happens per-request.

What breaks if missing/wrong?

Wrong import path -> `ImportError` at startup. Missing route -> 404. Misordered routes can shadow intended patterns.

Common beginner mistakes

- Not namespacing included urlconfs (leading to `reverse()` name collisions).
- Putting catch-all route at top, causing later patterns unreachable.

urls.py (app-level)

What is it, literally?

Per-app url patterns that keep routing modular. Example `finance/urls.py`:

```
from django.urls import path  
from . import views  
  
app_name = 'finance'  
urlpatterns = [  
    path('', views.dashboard, name='dashboard'),  
    path('transactions/', views.transaction_list, name='transaction_list'),  
    path('transactions/<int:pk>/', views.transaction_detail,  
        name='transaction_detail'),  
]
```

Why does Django need it?

Separates concerns and keeps routing for each app localized.

How does it work internally?

Included into project-level urls; when delegated, resolver looks into this list as if it's the top-level patterns.

When loaded/used?

Imported at startup when project-level urls include it; pattern matching per-request.

What breaks if missing/wrong?

Mistyped view name or module path => `ImportError` at startup or 404 at request.

Common beginner mistakes

Forgetting to set `app_name` causing reverse namespacing surprises.

views.py

What is it, literally?

Python module with view callables (function-based or class-based). They accept `HttpRequest` and return `HttpResponse/JsonResponse`, use forms/serializers and ORM.

Example FBV:

```
from django.shortcuts import render, get_object_or_404
from .models import Transaction

def transaction_list(request):
    qs = Transaction.objects.filter(user=request.user).select_related('category')
    [:50]
    return render(request, 'finance/transaction_list.html', {'transactions': qs})
```

Example CBV:

```
from django.views.generic import ListView

class TransactionListView(ListView):
    model = Transaction
    template_name = 'finance/transaction_list.html'
    paginate_by = 50

    def get_queryset(self):
        return
        Transaction.objects.filter(user=self.request.user).select_related('category')
```

Why does Django need it?

Views implement the application's request/response logic — they tie URL -> business logic -> presentation.

How does it work internally?

URL resolver returns view callable; for CBV, `as_view()` returns a function that constructs a view instance and calls `dispatch()`. The view receives `HttpRequest`, uses ORM/forms, and returns `HttpResponse`.

When is it loaded/used?

Views are imported when `URLConf` is imported at startup (so syntax/runtime errors in views can break startup), and executed per-request.

What breaks if missing/wrong?

Syntax error in `views.py` -> `ImportError` at startup. Returning wrong object type (not `HttpResponse`) -> `TypeError` at runtime. Unhandled exceptions propagate to 500 unless middleware handles.

Common beginner mistakes

Putting heavy business logic in view functions rather than service layer; forgetting to restrict data by `user` causing data leaks.

models.py

What is it, literally?

Python classes inheriting from `django.db.models.Model` declaring fields/relationships, Meta options, managers, methods. Example:

```
from django.db import models

class Transaction(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE, related_name='transactions')
    amount = models.DecimalField(max_digits=12, decimal_places=2)
    date = models.DateField()
    category = models.ForeignKey('Category', on_delete=models.PROTECT)
    description = models.TextField(blank=True)

    class Meta:
        indexes = [models.Index(fields=['user', 'date'])]
```

Why does Django need it?

Models are the canonical data schema for ORM mapping; they drive migrations and admin.

How does it work internally?

Model class creation runs through `ModelBase` metaclass that collects fields and builds model's `_meta`. At migrate time Django uses this metadata to create/alter DB schema.

When is it loaded/used?

Imported at startup when app registry loads; queries executed at runtime per-request.

What breaks if missing/wrong?

Misspell field names => `AttributeError` at access. Incorrect `on_delete` choices cause cascade issues.

Missing migrations if you change models without running `makemigrations` -> runtime DB mismatch.

Common beginner mistakes

Relying only on Python-level validation and not enforcing DB-level constraints (i.e., leaving NOT NULL when you need it).

admin.py

What is it, literally?

Module registering models with Django admin using `ModelAdmin` classes for customization. Example:

```
from django.contrib import admin
from .models import Transaction, Category

@admin.register(Transaction)
class TransactionAdmin(admin.ModelAdmin):
    list_display = ('id', 'user', 'amount', 'date', 'category')
    list_filter = ('category', 'date')
    search_fields = ('description',)
```

Why does Django need it?

Provides an out-of-the-box admin UI for CRUD on registered models.

How does it work internally?

`admin.site` registers model options and dynamically generates `ModelAdmin` views/forms; admin URLs are added to project urls with `admin.site.urls`.

When is it loaded/used?

`admin.py` is imported when `django.contrib.admin` is initialized (startup) if `INSTALLED_APPS` includes the app and the admin is enabled.

What breaks if missing/wrong?

Bad admin configuration can raise exceptions at startup or when accessing admin pages.

Common beginner mistakes

Leaving admin open to public without proper `ALLOWED_HOSTS` or admin hardening.

apps.py

What is it, literally?

`AppConfig` class that configures app name and `ready()` hook. Example:

```
from django.apps import AppConfig

class FinanceConfig(AppConfig):
    name = 'finance'
    verbose_name = 'Finance'

    def ready(self):
        import finance.signals # register signal handlers safely
```

Why does Django need it?

Registers app metadata in the app registry and provides startup hook to connect signals or perform app-level initialization.

How does it work internally?

`django.setup()` iterates `INSTALLED_APPS`, imports `AppConfig` classes, and calls `ready()` after all apps loaded.

When is it loaded/used?

At process startup during `django.setup()`.

What breaks if missing/wrong?

Errors in `ready()` (heavy DB calls or import cycles) cause startup failure.

Common beginner mistakes

Executing DB queries in `ready()` (before DB is ready / in management commands) or causing circular imports.

middleware.py (or custom middleware classes)

What is it, literally?

Middleware are callables/classes placed in `MIDDLEWARE` list; they can inspect/modify requests/responses. Example custom middleware:

```
class RequestTimerMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.time()
        response = self.get_response(request)
        duration = time.time() - start
        response['X-Elapsed'] = str(duration)
        return response
```

Why does Django need it?

Centralized cross-cutting concerns (security, sessions, authentication, logging, CORS, etc.).

How does it work internally?

Django wraps the view callable with middleware chain: each middleware receives `get_response` and returns a callable. On request phase middleware code runs top-down; on the way back response phase runs bottom-up.

When is it loaded/used?

Middleware classes are imported at startup, and their `__init__` called; `__call__` executed per-request.

What breaks if missing/wrong?

Misordered middleware can break auth, sessions, or CSRF. Heavy middleware can significantly increase request latency.

Common beginner mistakes

Writing blocking I/O or expensive operations in middleware and causing performance degradation.

forms.py

What is it, literally?

Modules defining `django.forms.Form` or `ModelForm` to validate and clean input. Example:

```
from django import forms
from .models import Transaction

class TransactionForm(forms.ModelForm):
    class Meta:
        model = Transaction
        fields = ['amount', 'date', 'category', 'description']

    def clean_amount(self):
        amt = self.cleaned_data['amount']
```

```
if amt == 0:
    raise forms.ValidationError("Amount cannot be zero")
return amt
```

Why does Django need it?

Centralizes input validation and conversion for HTML forms.

How does it work internally?

When view receives POST, instantiate form with POST data and call `form.is_valid()`; form runs field validators and `clean()` chain, populating `cleaned_data`.

When is it loaded/used?

Imported in views and used per-request at form submission time.

What breaks if missing/wrong?

Invalid forms/validation errors result in `form.is_valid()` False; if you forget to check, you might save invalid data.

Common beginner mistakes

Trusting `form.is_valid()` implicitly without calling it; forgetting to call `form.save()` with `commit=False` when you need to set extra fields.

wsgi.py

What is it, literally?

Module exposing WSGI application callable used by WSGI servers. Example:

```
import os
from django.core.wsgi import get_wsgi_application
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'finance_tracker.settings')
application = get_wsgi_application()
```

Why does Django need it?

It gives a standard WSGI callable to run Django in production (Gunicorn, uWSGI).

How does it work internally?

`get_wsgi_application` returns a WSGI app that wraps Django request handling; the server calls `application(environ, start_response)`.

When is it loaded/used?

At process start by WSGI server.

What breaks if missing/wrong?

Wrong settings module or errors in this file prevent the WSGI server from starting.

Common beginner mistakes

Running wsgi app directly in dev instead of runserver (lack of autoreload), or mismatching ASGI/WSGI expectations.

asgi.py

What is it, literally?

Module exposing ASGI application callable for async servers (Uvicorn, Daphne). Example:

```
import os
from django.core.asgi import get_asgi_application
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'finance_tracker.settings')
application = get_asgi_application()
```

Why does Django need it?

Supports async views, websockets and long-lived connections when using ASGI servers and channels.

How does it work internally?

ASGI scope and events passed to application; `get_asgi_application` returns a callable compatible with ASGI standard.

When is it loaded/used?

At process start by ASGI server.

What breaks if missing/wrong?

Using async features but starting WSGI server -> errors or no websocket support.

Common beginner mistakes

Blocking code inside async views (e.g., synchronous DB or external calls) without running them in thread pool.

templates/ (folder)

What is it, literally?

Directory tree containing DTL files. Django loads them using template loaders configured in `TEMPLATES` setting. Example path: `templates/finance/transaction_list.html`

Why does Django need it?

Provides view presentation layer for server-rendered pages.

How does it work internally?

Template loaders find files; template engine parses into nodes; render compiles nodes, resolves variables using context processors + view context, returns a string.

When is it loaded/used?

Loaded on demand per-request (or cached compiled template between requests if template caching enabled).

What breaks if missing/wrong?

`TemplateDoesNotExist` when `render()` called or wrong context variables causing `KeyError` in templates if strict config set.

Common beginner mistakes

Placing templates in app directory but not having app in `INSTALLED_APPS` or `TEMPLATE_DIRS` misconfigured.

static/ (folder)

What is it, literally?

CSS, JS, image assets stored under `static/` in apps or project-level static dirs. `STATICFILES_DIRS` lists dev directories; `STATIC_ROOT` is target for `collectstatic`.

Why needed?

Separates assets from templates and enables optimized serving in production (CDN or web server).

How it works internally?

During development, static files served by `django.contrib.staticfiles`. In production, `collectstatic` gathers assets into `STATIC_ROOT` for Nginx/WhiteNoise/CloudFront.

When used?

Per-request when template uses `{% static 'css/site.css' %}` tag to generate URL.

What breaks if missing/wrong?

Missing static files leads to 404 and broken UI.

Common mistake

Forgetting collectstatic on deploy or relying on runserver static serving in prod.

media/ (folder)

What is it, literally?

Directory where user-uploaded files are stored; configured by `MEDIA_ROOT` and served at `MEDIA_URL`.

Why needed?

Stores files (uploads) separate from static assets and code.

How it works internally?

`FileField/ImageField` stores path in DB; uploaded files saved to storage backend (local disk or S3) at save time.

When used?

At runtime when users upload; serve either by dev server in DEBUG or via web server/cloud storage in prod.

What breaks if missing/wrong?

File upload error; missing `MEDIA_ROOT` -> can't save files; wrong permissions -> write errors.

Common mistake

Serving media via runserver in production or forgetting to configure object storage and signed URLs for privacy.

services.py [GAP FILLED – YOUR project-specific file]

What is it, literally?

A module containing business logic functions that views call instead of touching ORM directly.

YOUR `finance/services.py`:

```
"""
Service layer for financial transactions.
Views call these functions – never ORM directly.
This ensures business logic is centralized and testable.
"""

from django.core.exceptions import ValidationError, PermissionDenied
from django.shortcuts import get_object_or_404
from .models import Transaction, Category, Budget, Notification

def create_transaction(user, cleaned_data):
    """Create a new transaction for the given user."""
    transaction = Transaction(
```

```

        user=user,
        category=cleaned_data['category'],
        amount=cleaned_data['amount'],
        currency=cleaned_data['currency'],
        date=cleaned_data['date'],
        description=cleaned_data.get('description', ''),
        receipt=cleaned_data.get('receipt'),
    )
    transaction.save() # full_clean() called inside save()

    if transaction.type == 'EXPENSE':
        _check_budget_overrun(user, transaction)
    return transaction

def update_transaction(user, transaction_id, cleaned_data):
    """Update an existing transaction. Enforces ownership."""
    transaction = get_object_or_404(Transaction, pk=transaction_id)
    if transaction.user_id != user.id:
        raise PermissionDenied('You do not have permission to edit this transaction.')
    # ... update fields ...
    transaction.save()
    return transaction

def delete_transaction(user, transaction_id):
    """Delete a transaction. Enforces ownership."""
    transaction = get_object_or_404(Transaction, pk=transaction_id)
    if transaction.user_id != user.id:
        raise PermissionDenied('You do not have permission to delete this transaction.')
    transaction.delete()

def get_user_transactions(user, filters=None):
    """Retrieve transactions with optional filters."""
    qs = Transaction.objects.filter(user=user).select_related('category', 'currency')
    if filters:
        if filters.get('type'):
            qs = qs.filter(type=filters['type'])
        if filters.get('category_id'):
            qs = qs.filter(category_id=filters['category_id'])
        if filters.get('date_from'):
            qs = qs.filter(date__gte=filters['date_from'])
        if filters.get('date_to'):
            qs = qs.filter(date__lte=filters['date_to'])
    return qs

```

Why does Django need this?

Django doesn't require it — **YOUR project chose this pattern**. It keeps views thin (orchestration only) and business logic centralized. Benefits:

- **Testable:** Test business logic without HTTP request/response.
- **Reusable:** Same logic used by views and future API endpoints (DRF).

- **Security:** Ownership checks (`user_id != user.id`) are enforced in one place.

How does it work?

Views call service functions, passing `request.user` and `form.cleaned_data`. The service handles ORM operations, validation, and side effects (like budget overrun notifications).

When is it loaded?

Imported by `views.py` at startup; executed per-request.

Common beginner mistake

Bypassing the service layer and doing ORM in views — this duplicates ownership checks and business rules.

context_processors.py [GAP FILLED – YOUR project-specific file]

What is it, literally?

A module with functions that inject variables into **every** template context automatically.

YOUR `finance/context_processors.py`:

```
from .models import Notification

def notification_count(request):
    """Inject unread notification count into every template."""
    if request.user.is_authenticated:
        count = Notification.objects.filter(user=request.user,
is_read=False).count()
        return {'unread_notification_count': count}
    return {'unread_notification_count': 0}
```

Why does Django need this?

So you don't have to pass `unread_notification_count` manually in every view's context. Any template can use `{{ unread_notification_count }}` — including `base.html` nav.

How does it work?

Registered in `TEMPLATES[0]['OPTIONS']['context_processors']` in `settings.py`. On every `render()` call, Django runs each context processor and merges returned dicts into the template context.

When loaded?

The function is called per-request (every time a template is rendered).

What breaks if wrong?

Heavy DB queries here slow down **every page**. Missing registration in settings -> variable undefined in templates.

Common beginner mistake

Running expensive queries in context processors without caching — this runs on every single page load.

Signals [GAP FILLED – YOUR project uses signals in core/signals.py]

What are they, literally?

Django signals are a pub/sub mechanism that lets decoupled components react to events. YOUR project uses `post_save` to auto-create UserProfile.

YOUR `core/signals.py`:

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.conf import settings
from .models import UserProfile

@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_user_profile(sender, instance, created, **kwargs):
    """Auto-create a UserProfile when a new User is created."""
    if created:
        UserProfile.objects.create(user=instance)
```

How are they connected?

In `core/apps.py`:

```
class CoreConfig(AppConfig):
    name = 'core'
    def ready(self):
        import core.signals # connects signal handlers at startup
```

Why?

Decouples User creation from Profile creation — you don't need to remember to create a profile manually.

When loaded?

`ready()` runs at startup -> imports `signals.py` -> `@receiver` decorators register handlers -> called on every `User.save()` where `created=True`.

What breaks?

- If you forget `import core.signals` in `ready()` -> signals never fire, profiles never created.
- If signal handler raises exception -> the triggering `save()` also fails (they share the same transaction by default).

Common beginner mistake

Importing signals at module level instead of in `ready()` — this can cause circular imports and duplicate registrations.

.env + python-decouple [GAP FILLED]

What is it, literally?

A `.env` file storing environment variables, read by `python-decouple` in `settings.py`.

YOUR `.env`:

```
SECRET_KEY=your-secret-key-here
DEBUG=True
DATABASE_URL=postgres://user:pass@localhost:5432/finance_tracker
RESEND_API_KEY=re_xxx
```

YOUR `settings.py` reads it:

```
from decouple import config
SECRET_KEY = config('SECRET_KEY')
DEBUG = config('DEBUG', default=False, cast=bool)
```

Why?

- **Security:** SECRET_KEY and DB passwords never committed to Git.
- **Portability:** Different values in dev/staging/prod without changing code.
- **12-factor app compliance:** Config stored in environment.

What breaks?

Missing `.env` or missing variable -> `UndefinedValueError` at startup.

Common beginner mistake

Committing `.env` to Git. Always add `.env` to `.gitignore`.

requirements.txt [GAP FILLED — YOUR project's dependencies explained]

Package	What it does in YOUR project
Django	The web framework
psycopg2-binary	PostgreSQL database driver
dj-database-url	Parses DATABASE_URL env var into Django DATABASES config
python-decouple	Reads .env file and provides config() function
gunicorn	Production WSGI server (replaces runserver)
whitenoise	Serves static files directly from Django in production
django-allauth	Google OAuth / social authentication
Pillow	Image processing for ImageField
resend	Email delivery SDK for budget alert notifications

CHAPTER 3 — ALL DJANGO COMMANDS (detailed internals & effects)

For each command: what it does internally, when to run it, what changes on disk/DB, and typical errors if misused.

```
django-admin startproject projectname .
```

What it does internally

Creates project directory structure with `manage.py`, `settings.py`, `urls.py`, `wsgi.py`, `asgi.py`, and `__init__.py`.

When to run

Once, at the very beginning of a new Django project.

What changes on disk/DB

Creates files on disk only. No DB changes.

Common errors

Running inside an existing project or wrong directory.

```
python manage.py startapp <appname>
```

What it does internally

Creates a new directory named after the app with boilerplate files:

```
finance/
  __init__.py          # makes finance a Python package
  admin.py             # register models in Django admin
  apps.py              # AppConfig subclass for app metadata & ready() hook
  migrations/          # migration package
    __init__.py        # marks migrations folder as package (empty initially)
  models.py            # place model classes here
  tests.py             # test cases for this app
  views.py             # view callables / CBVs for this app
```

Minimal content examples (what's inside by default):

apps.py:

```
from django.apps import AppConfig
class FinanceConfig(AppConfig):
    name = 'finance'
```

models.py:

```
from django.db import models
# empty scaffolding ready for your model classes
```

migrations/init.py:

```
# empty - migration system will add files here after makemigrations
```

It does NOT modify settings.py, INSTALLED_APPS, or any existing files — it only creates files on disk.

When would you run it?

When you are adding a new feature domain or bounded context and want modularity (e.g., transactions, reports, accounts). Typical workflow: create the app directory first, then add models/views/templates, add to INSTALLED_APPS, then makemigrations/migrate.

What changes on disk?

A new directory with the files listed above appears in the project workspace. Nothing else is changed (no settings update, no DB changes) until you edit files and run management commands.

What you MUST do next (common follow-ups)

1. Add the app to INSTALLED_APPS in settings.py (or use the AppConfig dotted path): `INSTALLED_APPS += ['finance.apps.FinanceConfig']` Without this, Django will not register models, admin,

migrations, templates/static lookups for this app.

2. Create models in `finance/models.py` and run: `python manage.py makemigrations finance python manage.py migrate`
3. Add urls for the app (`finance/urls.py`) and include them in project-level `urls.py`: `path('finance/', include('finance.urls', namespace='finance'))`

When is the app code loaded/used?

The created files are imported when Django loads the app registry (`django.setup()`) — typically at process startup (`runserver` / `gunicorn` / `manage.py` commands). But nothing in the DB or runtime changes until you add code and run migrations or use the app's views.

What breaks if you forget to do the follow-ups?

- If you don't add the app to `INSTALLED_APPS`:
 - Model classes won't be registered; `makemigrations` may not detect model changes.
 - Admin registration in `admin.py` won't show in the admin.
 - app-level templates/static may not be found by the app-specific finders.
- If you create an app name that shadows a Python stdlib module or an existing package (e.g., naming app "email" or "json"), you'll get import errors at startup.

Common beginner mistakes

- Forgetting to add the new app to `INSTALLED_APPS` (most common).
- Naming the app the same as a top-level package in your repo (causes import shadowing).
- Running `startapp` inside an unexpected directory (creating nested apps or wrong relative import paths).
- Assuming `startapp` runs `makemigrations/migrate` — it does not. You must create models and then run `makemigrations` and `migrate`.

Quick checklist after `startapp`

- ☐ Add app to `INSTALLED_APPS`
- ☐ Create models and run `makemigrations + migrate`
- ☐ Add `app.urls` and include them in project `urls.py`
- ☐ Register any models in `admin.py` if admin UI wanted
- ☐ Add templates/static folders as needed (`templates/finance/`, `static/finance/`)

Example developer sequence (concrete)

```
# 1. Create app:
python manage.py startapp finance

# 2. Register in settings.py:
# INSTALLED_APPS += ['finance.apps.FinanceConfig']

# 3. Add a model in finance/models.py, then:
python manage.py makemigrations finance
python manage.py migrate
```

```
# 4. Create finance/urls.py, wire in project urls.py, implement views/templates.
```

```
python manage.py runserver
```

What it does internally

- Calls the runserver management command which:
 - Calls `django.setup()` (loads settings, installed apps, app registry, `AppConfig.ready()`).
 - Sets up autoreload (watching Python source files).
 - Starts a development WSGI-compatible server (simple, single process/threaded) or ASGI dev server for async if configured.

What to run it for

Local dev and quick testing.

What changes on disk/DB

None (just process executes). Note: some code triggered on startup may perform side effects (e.g., signal registration).

Common errors

`ImportError` due to bad settings or missing env vars; port already in use; DEBUG-dependent code failing in production.

```
python manage.py makemigrations
```

What it does internally

- Compares current model state (in-app models.py and model `_meta`) with the "migration graph" (migrations files present) to detect changes.
- Generates migration files in `app/migrations/*_auto_*.py` containing Operation objects (`AddField`, `CreateModel`, etc.)

What to run it for

After adding/changing models to create migration scripts.

What changes on disk/DB

Creates new migration Python files on disk (**no DB changes yet**).

Common errors

Model import errors preventing makemigrations; changes that can't be autodetected (complex operations) require manual edits.

```
python manage.py migrate
```

What it does internally

- Reads migration graph, finds unapplied migrations, runs their operations in order.
- Each migration operation executes SQL or Python (`RunPython`) against DB using `schema_editor`.
- Records applied migrations in `django_migrations` table (applied migrations list).

What to run it for

Apply schema and data migrations to the DB when deploying or developing after makemigrations.

What changes on disk/DB

Alters DB schema (`CREATE TABLE`, `ALTER TABLE`, `CREATE INDEX`) and inserts migration records in `django_migrations`.

Common errors

DB permissions error, locked tables, incompatible state (migrations out of sync), apply order conflicts.

```
python manage.py showmigrations
```

What it does

Lists migrations for apps and marks which are applied `[X]` vs unapplied `[ ]`.

When to run

Check migration state across environments.

What it changes

Nothing.

Common issues

Migrations applied in DB but not present on disk or vice versa; divergence signals.

```
python manage.py sqlmigrate app_name migration_number
```

What it does

Compiles a single migration's operations into SQL for the configured DB backend; prints it so you can preview.

When to run

Inspect generated SQL, especially for large table changes.

What it changes

Nothing on DB; pure preview.

Common mistake

Relying on default SQL without reviewing impact on production (locks, table rewrites).

```
python manage.py createsuperuser
```

What it does

Interactive prompt that creates an `auth.User` with `is_staff/is_superuser` flags; password hashed via configured password hasher.

When to run

Initial setup for admin access in dev or staging.

What it changes

Inserts rows in `auth_user` table.

Common errors

Using default weak password policies; running in non-interactive CI needs `--noinput` with pre-seeded env variables.

```
python manage.py shell
```

What it does

Bootstraps Django and drops into an interactive Python shell with Django configured; if `django-extensions` installed, `shell_plus` loads models automatically.

When to run

Ad-hoc ORM testing/debugging, executing scripts with Django context.

What changes

No changes unless you execute DB operations in shell.

Common pitfalls

Running destructive commands on production DB accidentally if connected.

```
python manage.py collectstatic
```

What it does

Uses staticfiles finders (app static directories, `STATICFILES_DIRS`) to collect and copy static assets into `STATIC_ROOT` or a storage backend (S3). Optional storage backends can fingerprint/narrow files (`ManifestStaticFilesStorage`).

When to run

Before serving static assets in production or when deploying.

What changes

Copies files into `STATIC_ROOT`; may write manifest (e.g., `staticfiles.json`) when using `ManifestStaticFilesStorage`.

Common errors

Missing `STATIC_ROOT` or permissions error; broken manifest if storage expects hashed filenames but templates reference originals (use `{% static %}`).

```
python manage.py test
```

What it does

Discovers tests per Django testing policy (unittest-based), creates test database(s), runs migrations (or uses serialized test DB), runs test suite, then tears down test DB.

When to run

CI, pre-commit checks, local testing.

What changes

Creates temporary test DB and applies migrations; no persistent changes when complete.

Common pitfalls

Tests relying on local uncommitted data, slow tests due to heavy fixtures, using production DB tests accidentally.

```
python manage.py dumpdata
```

What it does

Serializes DB rows into fixture format (JSON by default), respecting natural keys if configured.

When to run

Backups, seed data exports.

What changes

Writes fixture files to disk; no DB change.

Common issues

Dumping large DB causing huge fixture files; exposing sensitive data.

`python manage.py loaddata`

What it does

Reads fixture(s) and deserializes them into model instances, inserting or updating DB rows.

When to run

Seeding dev data, loading fixtures in tests.

What changes

Modifies DB.

Common mistakes

Loading fixtures incompatible with current schema; duplicates or PK conflicts.

Additional admin/dev commands

- `python manage.py dbshell` — opens DB shell using configured DB credentials (psql for Postgres).
 - `python manage.py check` — runs project checks for common misconfigurations.
 - `python manage.py showmigrations` — list migrations with applied/unapplied status.
-

QUICK TROUBLESHOOTING CHEAT SHEET (commands + typical errors)

Error	Likely Cause	Fix
<code>ImportError</code> at runserver	Bad settings or missing dependency	Check <code>DJANGO_SETTINGS_MODULE</code> , install packages
<code>OperationalError: no such column</code>	Forgot to run <code>migrate</code>	Run <code>python manage.py migrate</code>
<code>TemplateDoesNotExist</code>	Wrong template path or missing app in <code>INSTALLED_APPS</code>	Check path, add app
<code>ModuleNotFoundError</code> on startup	Incorrect import or <code>PYTHONPATH</code>	Fix import, check virtualenv
<code>Permission denied</code> writing to <code>STATIC_ROOT/MEDIA_ROOT</code>	Filesystem permissions	Fix perms or storage config
Migrations applied in prod but not in repo	Ensure migrations committed to VCS	Commit migration files

CHAPTER 4 — ORM MASTERY

The Object-Relational Mapper (ORM) is how you interact with the database using Python instead of writing raw SQL.

4.1 Database Configuration (`settings.DATABASES`)

What is it, literally?

A dictionary telling Django how to connect to the database. YOUR project's config:

```
import dj_database_url
from decouple import config

DATABASES = {
    'default': dj_database_url.config(default=config('DATABASE_URL'))
}
```

Why does Django need it?

Without it, Django cannot store or retrieve models, apply migrations, or run authentication.

How does it work internally?

At startup, Django parses this dictionary to set up connection configurations. It doesn't actually connect to the database until the first query is made (or a migration is run). The `dj_database_url` package parses a URL like `postgres://user:pass@localhost:5432/db` into the dictionary Django expects.

What breaks if wrong?

`OperationalError` (connection refused, bad password, database does not exist) when trying to run queries or runserver/migrations.

4.2 Making Queries (The Basics)

What are they, literally?

Python methods on `Model.objects` that generate and execute SQL.

Common Queries

1. Create a record (`INSERT`)

```
# Returns the saved instance
cat = Category.objects.create(user=request.user, name="Groceries", type="EXPENSE")
```

2. Fetch all records (**SELECT ***)

```
all_cats = Category.objects.all() # Returns a QuerySet
```

3. Filter records (**SELECT ... WHERE ...**)

```
expenses = Category.objects.filter(type="EXPENSE", user=request.user)
```

4. Fetch a single record (**SELECT ... LIMIT 1**)

```
# Raises Category.DoesNotExist if 0 found
# Raises Category.MultipleObjectsReturned if >1 found
cat = Category.objects.get(id=1)
```

5. Update a single record (**UPDATE**)

```
cat = Category.objects.get(id=1)
cat.name = "Supermarket"
cat.save() # Runs UPDATE
```

6. Delete a record (**DELETE**)

```
cat = Category.objects.get(id=1)
cat.delete() # Runs DELETE (and cascades if configured)
```

4.3 QuerySets and Laziness

What is it?

A **QuerySet** is a representation of a database query. It is **lazy**, meaning it doesn't actually hit the database until you evaluate it (e.g., by iterating over it, slicing it, or converting it to a list).

Why?

Allows you to chain filters together without making multiple database calls.

```
# NO DB CALL YET
qs = Transaction.objects.filter(user=request.user)

# STILL NO DB CALL
```

```
qs = qs.filter(type='EXPENSE')

# DB CALL HAPPENS HERE (iteration)
for tx in qs:
    print(tx.amount)
```

How to evaluate a QuerySet

- Iteration (`for tx in qs:`)
- Slicing with step or evaluation (`list(qs)`)
- Pickling/Caching
- `repr()` (when printing in shell)
- Methods like `len(qs)` (but use `qs.count()` to do `SELECT COUNT(*)`)
- Evaluating as a boolean (`if qs:` - but use `qs.exists()` instead)

4.4 Advanced Querying (`F()` expressions, `Q()` objects, Aggregation)

#`F()` Expressions [GAP FILLED]

What: Allows you to reference the value of a model field directly in the database without pulling it into Python. **When to use:** Race conditions, atomic updates, or comparing fields in the same row.

```
from django.db.models import F

# Compare two fields on the same row:
# "Find budgets where spent is greater than limit_amount"
overrun = Budget.objects.filter(spent__gt=F('limit_amount'))

# Atomic updates (avoids race condition):
Budget.objects.filter(id=1).update(spent=F('spent') + 10)
```

#`Q()` Objects

What: Used for complex queries with `OR` or `NOT`. Standard `.filter()` only does `AND`.

```
from django.db.models import Q

# Find transactions that are EITHER Health OR Education
txs = Transaction.objects.filter(
    Q(category__name='Health') | Q(category__name='Education')
)

# NOT queries
txs = Transaction.objects.filter(~Q(type='INCOME'))
```

Aggregation & Annotation

Aggregate: Returns a dictionary of computed values for the *entire* QuerySet (**SUM**, **AVG**, **COUNT**).

```
from django.db.models import Sum
# SELECT SUM(amount) FROM transaction
total = Transaction.objects.filter(user=user).aggregate(total_spent=Sum('amount'))
print(total['total_spent'])
```

Annotate: Adds computed values to *each item* in the QuerySet.

```
from django.db.models import Count
# Adds a 'tx_count' attribute to each Category object
cats = Category.objects.annotate(tx_count=Count('transactions'))
for c in cats:
    print(c.name, c.tx_count)
```

4.5 The N+1 Problem (select_related & prefetch_related)

What is it?

The N+1 problem occurs when you query a list of items (1 query), and then access a related object for each item, triggering a new query per item (N queries).

Bad Example (N+1):

```
txs = Transaction.objects.all() # 1 query
for tx in txs:
    print(tx.category.name)      # N queries (one for each transaction's category)
```

The Fix: **select_related** (For ForeignKey / OneToOne)

Does a SQL **JOIN** to get the related data in the initial query.

```
# 1 query total (LEFT OUTER JOIN)
txs = Transaction.objects.select_related('category').all()
for tx in txs:
    print(tx.category.name) # No extra query!
```

The Fix: **prefetch_related** (For ManyToMany / Reverse ForeignKey)

Does a separate query for the related items and joins them in Python.

```
# 2 queries total:
# 1. SELECT * FROM category
# 2. SELECT * FROM transaction WHERE category_id IN (...)
cats = Category.objects.prefetch_related('transactions').all()
for cat in cats:
    for tx in cat.transactions.all(): # No extra queries!
        print(tx.amount)
```

4.6 Model Deep Dive [GAP FILLED]

#`on_delete` behaviors

When you define a `ForeignKey`, you MUST specify `on_delete`. This tells Django what to do when the referenced object is deleted.

- `models.CASCADE`: Delete the object containing the `ForeignKey`. (If User is deleted, delete their Budgets).
- `models.PROTECT`: Prevent deletion of the referenced object. (You cannot delete a Category if it has Transactions). *Used in YOUR Transaction model.*
- `models.SET_NULL`: Set the `ForeignKey` to null (requires `null=True`).
- `models.SET_DEFAULT`: Set to default value.

#`related_name`

The name to use for the reverse relation from the related object back to this one.

```
class Category(models.Model):
    user = models.ForeignKey(User, related_name='categories')

# Usage:
user.categories.all() # Instead of user.category_set.all()
```

#`class Meta`

Provides model-level metadata.

```
class Meta:
    ordering = ['-date', '-created_at'] # Default order for QuerySets
    verbose_name_plural = 'Categories' # Admin panel name
    unique_together = ['user', 'category', 'month'] # DB constraint
    indexes = [models.Index(fields=['user', 'date'])] # DB Index for speed
```

#`clean()` and `save()`

Models can override these to add custom logic.

- `clean()`: Used for model validation. Must be called explicitly (or by a `ModelForm`). It does NOT run automatically on `.save()`.
- `save()`: Used to modify data right before it goes to the DB.

YOUR Transaction model does both:

```
def save(self, *args, **kwargs):
    # Auto-set type from category before every save
    if self.category_id:
        self.type = self.category.type
    self.full_clean() # Forces clean() to run on save!
    super().save(*args, **kwargs)
```

CHAPTER 5 — TEMPLATES DEEP DIVE

5.1 Minimal template file (DTL) — real code and literal explanation

File: `templates/finance/transaction_list.html`

```
{% extends "base.html" %}
{% load static %}

{% block content %}
    <h1>Transactions</h1>
    <ul>
        {% for tx in transactions %}
            <li>{{ tx.date }} - {{ tx.category.name }} - {{ tx.amount }}</li>
        {% empty %}
            <li>No transactions found.</li>
        {% endfor %}
    </ul>

    {% if request.user.is_superuser %}
        <p>Admin alert!</p>
    {% endif %}

    <link rel="stylesheet" href="{% static 'css/finance.css' %}">
{% endblock %}
```

What each piece does, literally:

- `{% extends "base.html" %}` — Tells Django Template Language (DTL) to inherit `base.html` (template inheritance).
- `{% load static %}` — Loads the static template tag library so you can use `{% static 'path' %}`.
- `{% block content %} ... {% endblock %}` — Defines/overrides a named region from the base template.

- `{{ tx.amount }}` — Variable interpolation: inserts the value, auto-escaping HTML.
- `{% for ... %} ... {% empty %} ... {% endfor %}` — Loop with a fallback if the list is empty.
- `{% if ... %}` — Condition tag.
- `{% static 'css/finance.css' %}` — Returns the URL path for a static asset.

DTL vs Jinja2 — key differences (explicit)

- DTL auto-escapes variables by default; Jinja2 can be different depending on config.
- DTL has fewer built-in Python-like expressions (no arbitrary function calls like `{{ tx.calculate() }}`). Jinja allows more logic.
- DTL template tag libraries are pluggable.
- Django-specific tags (`{% url %}`, `{% static %}`, `{% csrf_token %}`) are DTL-first.
- **Rule:** Prefer DTL for Django apps unless you need Jinja features. DTL encourages keeping logic out of templates.

5.2 Full render flow (code + literal handoffs)

Sequence and minimal code at each step:

View (views.py)

```
def tx_detail(request, pk):
    tx = get_object_or_404(Transaction.objects.select_related('category'), pk=pk,
user=request.user)
    return render(request, 'finance/tx_detail.html', {'tx': tx})
```

Render flow (step-by-step):

1. Browser requests URL GET `/finance/transactions/42/`
2. URL resolver matches pattern and calls `tx_detail(request, pk=42)`
3. View builds context: `{'tx': tx}` where `tx` is a model instance (QuerySet executed here via `get_object_or_404`).
4. `render()` calls template loader to find `templates/finance/tx_detail.html` and `base.html`.
5. Template engine compiles template (cached) and renders with the context and context processors (e.g., `request`, `user`).
6. Inside template, `{% static 'css/finance.css' %}` is replaced with `/static/css/finance.css`.
7. HTML response sent back to browser; browser requests linked static CSS/JS files (separate HTTP requests) and renders page.

5.3 Context Processors [GAP FILLED]

What is it?

Functions that run before template rendering to inject variables into the context of *every* template.

How they work:

Configured in `settings.TEMPLATES['OPTIONS']['context_processors']`.

Standard ones:

- `request`: Injects the `request` object.
- `auth`: Injects `user` and `perms`.
- `messages`: Injects flash messages.

YOUR custom one (`finance/context_processors.py`):

```
def notification_count(request):
    if request.user.is_authenticated:
        count = Notification.objects.filter(user=request.user,
is_read=False).count()
        return {'unread_notification_count': count}
    return {'unread_notification_count': 0}
```

Now `{{ unread_notification_count }}` is available globally (used in your nav bar!).

5.4 Static files vs Media files — settings and literal differences

settings.py (real config)

```
# Static files (dev: project-owned CSS/JS/images)
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / "static"]          # development assets folder
STATIC_ROOT = BASE_DIR / "staticfiles"           # target for collectstatic in
production

# Media files (user uploads)
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / "media"
```

Literal meaning

- `STATICFILES_DIRS`: locations where `collectstatic` (and dev static finder) looks for your app/project static files.
- `STATIC_ROOT`: where `collectstatic` copies all static assets for production serving.
- `MEDIA_ROOT`: filesystem path where uploaded user files are saved.
- `MEDIA_URL`: URL prefix to serve uploaded files.

Where to put what

- Project CSS/JS/images you author — put under `static/`.
- User uploads (receipts, avatars) — saved via `FileField/ImageField` into `MEDIA_ROOT`.

5.5 "Add a new image to a page" — exact minimal changes (static image)

Goal: show a marketing image or icon in a template.

Files & actions:

- Place image file on disk: `static/img/finance-hero.png`
- Reference in template:

```
{% load static %}

```

Dev behavior: In `DEBUG=True`, Django serves it. `{% static %}` returns `/static/img/finance-hero.png`.

Production behavior: Run `collectstatic` to copy it to `STATIC_ROOT`. Nginx or WhiteNoise serves it.

5.6 "User uploads an image (receipt photo)" — full code and physical file path

Model (YOUR `Transaction`)

```
receipt = models.FileField(upload_to='receipts/%Y/%m/', blank=True, null=True)
```

Form (YOUR `TransactionForm`)

```
class TransactionForm(forms.ModelForm):
    class Meta:
        model = Transaction
        fields = ['receipt', ...]
```

Template

```
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Upload</button>
</form>
```

Crucial: `enctype="multipart/form-data"` is required for file uploads.

Where the file ends up physically: Given `MEDIA_ROOT = /path/media` and `upload_to='receipts/%Y/%m/'`, an upload on 2026-06-24 goes to: `/path/media/receipts/2026/06/receipt.pdf` Database stores the relative path: `'receipts/2026/06/receipt.pdf'`.

5.7 `collectstatic` — what it does to files on disk (concrete)

Command: `python manage.py collectstatic --noinput`

What it does, literally:

- Scans `STATICFILES_DIRS` and each app's `static/` directories.
- Copies each discovered file into `STATIC_ROOT`.
- If using `ManifestStaticFilesStorage`, generates hashed filenames (e.g. `main.3f2a1d.js`) and a manifest file `staticfiles.json` mapping original names to hashed names. This prevents stale caching in browsers.

WhiteNoise (simple production static serving in YOUR project):

- Pip install `whitenoise`.
- Add `'whitenoise.middleware.WhiteNoiseMiddleware'` to `MIDDLEWARE` right after `SecurityMiddleware`.
- WhiteNoise serves the files directly from Python, no separate Nginx required for simple deployments.

Summary table: Static vs Media

Feature	Static	Media
Who owns it	Developers	Users
Example	CSS, JS, logo.png	Receipts, Avatars
Settings	<code>STATICFILES_DIRS</code> , <code>STATIC_ROOT</code>	<code>MEDIA_ROOT</code> , <code>MEDIA_URL</code>
Served By	<code>collectstatic</code> -> WhiteNoise/Nginx	Web server direct / S3

CHAPTER 6 — AUTHENTICATION & AUTHORIZATION

6.1 Full login flow with concrete code

YOUR project uses `django-allauth` for social (Google) and local (Email/Password) login, but the underlying Django session auth concepts are identical. Let's look at standard Django login flow first.

Project URLs:

```
from django.contrib.auth import views as auth_views
urlpatterns = [
    path('accounts/login/',
    auth_views.LoginView.as_view(template_name='login.html'), name='login'),
]
```

What happens step-by-step (internals):

1. User submits POST to `/accounts/login/` with username/password + CSRF token.
2. `LoginView` calls `django.contrib.auth.authenticate(request, username, password)`.
3. `authenticate()` checks backend (e.g. `ModelBackend`), loads User, verifies password hash.
4. `login(request, user)` is called. It sets session data: `request.session[SESSION_KEY] = user.pk`.
5. Server responds with `Set-Cookie: sessionid=<key>`.
6. Subsequent request: browser sends `Cookie: sessionid=<key>`.
7. `SessionMiddleware` loads session data.
8. `AuthenticationMiddleware` reads `user_id` from session and sets `request.user` to the authenticated User.

Where is session stored?

Configured by `SESSION_ENGINE` (default: DB table `django_session`).

6.2 allauth & Google OAuth [GAP FILLED]

YOUR project uses `django-allauth`.

settings.py:

```
INSTALLED_APPS += [
    'django.contrib.sites',
    'allauth',
    'allauth.account',
    'allauth.socialaccount',
    'allauth.socialaccount.providers.google',
]
SITE_ID = 1
AUTHENTICATION_BACKENDS = [
    'django.contrib.auth.backends.ModelBackend',
    'allauth.account.auth_backends.AuthenticationBackend',
]
# Use email instead of username
ACCOUNT_AUTHENTICATION_METHOD = 'email'
ACCOUNT_EMAIL_REQUIRED = True
```

How allauth works: Instead of `LoginView`, requests hit `allauth` views. For Google:

1. User clicks "Login with Google".
2. Hits `allauth` social login URL -> redirects to Google.
3. Google returns to `allauth` callback URL with auth code.
4. `Allauth` fetches user email/profile, creates User if new, calls `login(request, user)`, sets session cookie.

6.3 Decorators: `@login_required`

views.py

```
from django.contrib.auth.decorators import login_required

@login_required(login_url='/accounts/login/')
def dashboard(request):
    # request.user is guaranteed to be an authenticated user
    return render(request, 'reports/dashboard.html')
```

Under the hood: It checks `request.user.is_authenticated`. If false, redirects to `login_url` with `?next=/dashboard/`.

6.4 Authorization & Ownership Checks [GAP FILLED]

Django has a permission system (`@permission_required`), but for SAAS/User-owned data (like YOUR Finance Tracker), **Object-Level Permissions / Ownership Checks** are more common.

YOUR `services.py` enforces this:

```
def update_transaction(user, transaction_id, cleaned_data):
    transaction = get_object_or_404(Transaction, pk=transaction_id)

    # CRITICAL SECURITY CHECK
    if transaction.user_id != user.id:
        raise PermissionDenied('You do not have permission to edit this transaction.')
    # ...
```

Always filter querysets by user (`Transaction.objects.filter(user=request.user)`) and check ownership before update/delete.

6.5 The Messages Framework [GAP FILLED]

What: One-time flash messages displayed to the user on the next page load.

In view:

```
from django.contrib import messages
messages.success(request, 'Transaction created successfully!')
```

In `base.html`:

```
{% if messages %}
  {% for message in messages %}
    <div class="alert alert-{{ message.tags }}">{{ message }}</div>
  {% endfor %}
{% endif %}
```

How it works: `MessageMiddleware` stores the message in the session/cookie. When the template iterates over `messages`, they are marked as read and deleted from storage.

CHAPTER 7 — MIDDLEWARE

7.1 What is it?

Hooks executed for every request/response to handle cross-cutting concerns. Request-phase executes top-down; response-phase executes bottom-up.

7.2 The MIDDLEWARE List & Execution Order

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'whitenoise.middleware.WhiteNoiseMiddleware', # static files
    'django.contrib.sessions.middleware.SessionMiddleware', # loads session
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware', # checks CSRF
    'django.contrib.auth.middleware.AuthenticationMiddleware', # loads user
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

ASCII Diagram:

```
-> SecurityMiddleware (request)
-> SessionMiddleware (request)
  -> CsrfViewMiddleware (request)
    -> AuthenticationMiddleware (request)
      -> URL Resolver -> View
    <- AuthenticationMiddleware (response)
  <- CsrfViewMiddleware (response)
<- SessionMiddleware (response)
<- SecurityMiddleware (response)
```

7.3 Why order matters

`SessionMiddleware` must run *before* `AuthenticationMiddleware`, because Auth needs `request.session` to find the user ID.

7.4 Built-in Middleware Deep Dive

a) SessionMiddleware

- **Request:** Looks up `sessionid` cookie, loads dict from DB, attaches to `request.session`.
- **Response:** If session changed, saves to DB and sets `Set-Cookie` header.

b) AuthenticationMiddleware

- **Request:** Uses `request.session` to get user ID, fetches User from DB (lazily), sets `request.user`.

c) CsrfViewMiddleware

- **Request:** For POST/PUT/DELETE, checks if CSRF token in POST data (`csrfmiddlewaretoken`) matches token in cookie. Raises 403 Forbidden if mismatched.

7.5 Custom Middleware

Example `RequestTimerMiddleware`:

```
import time

class RequestTimerMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response # Next middleware/view

    def __call__(self, request):
        # Code executed BEFORE view (top-down)
        start = time.perf_counter()

        response = self.get_response(request)

        # Code executed AFTER view (bottom-up)
        duration = (time.perf_counter() - start) * 1000
        response['X-Request-Duration-ms'] = f"{duration:.2f}"
        return response
```

Beginner Mistakes

- Doing heavy DB calls in middleware — it slows down *every single request* on the site!
- Misordering the list (e.g., putting custom auth before sessions).

CHAPTER 8 — FORMS DEEP DIVE [GAP FILLED]

Forms in Django handle HTML rendering, input parsing, and validation.

8.1 Regular Form vs ModelForm

- `forms.Form`: You define all fields manually. Use for login, contact forms, or filters.
- `forms.ModelForm`: Django creates the form from a Model. Use for CRUD operations.

YOUR `TransactionFilterForm` is a regular `Form`:

```
class TransactionFilterForm(forms.Form):
    type = forms.ChoiceField(choices=TYPE_CHOICES, required=False)
    date_from = forms.DateField(required=False, widget=forms.DateInput(attrs=
{'type': 'date'}))
```

YOUR `CategoryForm` is a `ModelForm`:

```
class CategoryForm(forms.ModelForm):
    class Meta:
        model = Category
        fields = ['name', 'type', 'description']
```

8.2 Rendering Forms in Templates

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }} <!-- Renders fields wrapped in <p> tags -->
    <button type="submit">Save</button>
</form>
```

Options: `form.as_p`, `form.as_table`, `form.as_div`. Or render manually for CSS frameworks:

```
<div class="form-group">
    <label>{{ form.amount.label }}</label>
    {{ form.amount }}
    {{ form.amount.errors }}
</div>
```

8.3 Customizing Widgets

Widgets control the HTML `<input>` type. YOUR `TransactionForm`:

```
class Meta:
    model = Transaction
    fields = ['category', 'amount', 'date', 'description']
    widgets = {
```

```

        'date': forms.DateInput(attrs={'type': 'date'}), # Renders HTML5 Date
        Picker
        'description': forms.Textarea(attrs={'rows': 3}),
        'amount': forms.NumberInput(attrs={'step': '0.01'}),
    }

```

8.4 Form Validation & Cleaning

When you call `form.is_valid()`, Django runs:

1. `to_python()`: Converts strings to Python types (e.g. "2026-06-24" -> `datetime.date`).
2. Field-specific validation (`clean_amount()`).
3. Form-wide validation (`clean()`).

YOUR `TransactionForm` has a custom field cleaner for receipts:

```

def clean_receipt(self):
    receipt = self.cleaned_data.get('receipt')
    if receipt:
        if receipt.size > 5 * 1024 * 1024: # 5 MB limit
            raise forms.ValidationError('File size must be under 5 MB.')
    return receipt

```

YOUR `BudgetForm` has a form-wide cleaner to normalize the month:

```

def clean_month(self):
    month = self.cleaned_data.get('month')
    if month:
        return month.replace(day=1) # Always store as 1st of month
    return month

```

8.5 `__init__` filtering (Crucial for Multi-Tenant apps)

If a user is adding a transaction, the `category` dropdown should **ONLY show their categories**.

YOUR `TransactionForm` solves this in `__init__`:

```

def __init__(self, *args, user=None, **kwargs):
    super().__init__(*args, **kwargs)
    if user:
        self.fields['category'].queryset = Category.objects.filter(user=user)

```

In `views.py`, you must pass the user:

```
form = TransactionForm(request.POST, user=request.user)
```

8.6 Handling Form Submissions in Views

```
def budget_create(request):
    if request.method == 'POST':
        form = BudgetForm(request.POST, user=request.user)
        if form.is_valid():
            # commit=False creates the object but doesn't save to DB yet
            budget = form.save(commit=False)
            budget.user = request.user # Set missing required field
            budget.save()
            return redirect('finance:budget_list')
        else:
            form = BudgetForm(user=request.user) # Empty GET form

    return render(request, 'finance/budget_form.html', {'form': form})
```

CHAPTER 9 — DRF BRIDGE

Django REST Framework (DRF) is the standard way to build JSON APIs in Django.

9.1 Side-by-side: server-rendered view vs DRF endpoint

Django template version (server-rendered)

```
# finance/views.py
def transaction_list(request):
    qs = Transaction.objects.filter(user=request.user).select_related('category')
    return render(request, 'finance/transaction_list.html', {'transactions': qs})
```

Builds QuerySet -> renders HTML with template.

DRF equivalent — Serializer + ViewSet (API)

```
# finance/api/serializers.py
from rest_framework import serializers

class TransactionSerializer(serializers.ModelSerializer):
    category_name = serializers.CharField(source='category.name', read_only=True)

    class Meta:
```

```
model = Transaction
fields = ('id', 'date', 'amount', 'category_name', 'receipt')
```

```
# finance/api/views.py
from rest_framework import viewsets, permissions

class TransactionViewSet(viewsets.ModelViewSet):
    serializer_class = TransactionSerializer
    permission_classes = [permissions.IsAuthenticated]

    def get_queryset(self):
        return
Transaction.objects.filter(user=self.request.user).select_related('category')
```

```
# finance/api/urls.py
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
router.register(r'transactions', TransactionViewSet, basename='transaction')
urlpatterns = router.urls
```

Line-by-line mapping:

- Template view builds QuerySet -> DRF ViewSet `get_queryset()` returns same QuerySet.
- Template context variables -> Serializer fields.
- Rendering template to HTML -> DRF serializes to JSON.
- `@login_required` -> `permission_classes = [IsAuthenticated]`.
- project urls -> DRF router registers viewset endpoints (`/api/transactions/`).

9.2 DRF building blocks — minimal code + what/why/how

Serializer (`ModelSerializer`)

- **What:** Converts model instances to JSON and validates JSON input for create/update.
- **Why:** Separates data shape/validation from views.

APIView vs ViewSet

- **APIView (low-level):** Gives full control; implement `get()`, `post()`.
- **ViewSet (high-level):** Maps actions (list/create/retrieve/update/destroy) to methods and integrates with routers. Use when you want RESTful resource endpoints quickly.

Router

- **What:** Creates standard REST endpoints automatically (`/api/transactions/`, `/api/transactions/1/`).

Permissions & Throttling

```
permission_classes = [IsAuthenticated] # Restricts access
```

Throttling rate-limits requests (e.g., 100/day). Set in `settings.REST_FRAMEWORK`.

Authentication (Session vs JWT)

- **Session Auth (Django Default):** Uses cookies/sessionid. Great for browser single-page-apps (React/Vue) hosted on the same domain. Requires CSRF.
- **JWT (JSON Web Token):** Stateless token sent in `Authorization: Bearer <token>` header. Great for mobile apps or external APIs. Requires `djangorestframework-simplejwt`.

9.3 What stays identical vs what's new

Stays identical:

- Models, ORM (`select_related`), Migrations.
- Core business logic (YOUR `services.py` can be called from DRF exactly as it is called from views).

New:

- Serializers (replaces Forms).
- ViewSets/APIViews (replaces Template Views).
- JSON Responses (replaces HTML Templates).

CHAPTER 10 — DEVELOPMENT WORKFLOWS & RECIPES

These are practical recipes showing the exact files and commands needed to accomplish common tasks.

10.1 Recipe 1: Add a User Profile (OneToOne with Image)

Goal: Create a `UserProfile` model linked to `User` to store a profile picture and preferred currency, and auto-create it when a User is created.

1) models.py (core/models.py)

```
from django.db import models
from django.conf import settings
from finance.models import Currency

class UserProfile(models.Model):
    user = models.OneToOneField(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE, related_name='profile')
    avatar = models.ImageField(upload_to='avatars/', null=True, blank=True)
```

```
preferred_currency = models.ForeignKey(Currency, on_delete=models.SET_NULL,
null=True, blank=True)

def __str__(self):
    return f"{self.user.username}'s Profile"
```

2) signals.py (core/signals.py)

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.conf import settings
from .models import UserProfile

@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_user_profile(sender, instance, created, **kwargs):
    if created:
        UserProfile.objects.create(user=instance)
```

3) apps.py (core/apps.py)

```
from django.apps import AppConfig

class CoreConfig(AppConfig):
    name = 'core'
    def ready(self):
        import core.signals # Wire up the signal
```

4) forms.py (core/forms.py)

```
from django import forms
from .models import UserProfile

class UserProfileForm(forms.ModelForm):
    class Meta:
        model = UserProfile
        fields = ['avatar', 'preferred_currency']
```

5) views.py (core/views.py)

```
from django.shortcuts import render, redirect
from django.contrib.auth.decorators import login_required
from .forms import UserProfileForm

@login_required
def profile_edit(request):
```

```
# The signal guarantees request.user.profile exists
if request.method == 'POST':
    form = UserProfileForm(request.POST, request.FILES,
instance=request.user.profile)
    if form.is_valid():
        form.save()
        return redirect('profile')
else:
    form = UserProfileForm(instance=request.user.profile)
    return render(request, 'core/profile.html', {'form': form})
```

6) templates (core/profile.html)

```
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Save</button>
</form>

{% if request.user.profile.avatar %}
    
{% endif %}
```

7) Run commands:

```
python manage.py makemigrations core
python manage.py migrate
```

10.2 Recipe 2: GET-based Search & Filter

Goal: Create a transaction list with a filter form (category and type) via **GET** parameters.

1) forms.py (finance/forms.py)

```
class TransactionFilterForm(forms.Form):
    TYPE_CHOICES = [('', 'All')] + Transaction.TYPE_CHOICES
    type = forms.ChoiceField(choices=TYPE_CHOICES, required=False)
    category = forms.ModelChoiceField(queryset=Category.objects.none(),
required=False, empty_label='All')

    def __init__(self, *args, user=None, **kwargs):
        super().__init__(*args, **kwargs)
        if user:
            self.fields['category'].queryset = Category.objects.filter(user=user)
```

2) views.py (finance/views.py)

```
def transaction_list(request):
    # Bind form to GET data
    form = TransactionFilterForm(request.GET, user=request.user)
    qs = Transaction.objects.filter(user=request.user).select_related('category',
    'currency')

    if form.is_valid():
        if form.cleaned_data.get('type'):
            qs = qs.filter(type=form.cleaned_data['type'])
        if form.cleaned_data.get('category'):
            qs = qs.filter(category=form.cleaned_data['category'])

    return render(request, 'finance/transaction_list.html', {'transactions': qs,
    'form': form})
```

3) Template (finance/transaction_list.html)

```
<!-- GET method allows bookmarking the search URL -->
<form method="get">
    {{ form.as_p }}
    <button type="submit">Filter</button>
</form>

<ul>
{% for tx in transactions %}
    <li>{{ tx.amount }}</li>
{% endfor %}
</ul>
```

CHAPTER 11 — DEPLOYMENT & PRODUCTION [GAP FILLED]

When moving from `runserver` to a real production environment (like Render, Heroku, or AWS), several things must change.

11.1 The Production Stack

In development:

- Web Server: `manage.py runserver`
- Database: SQLite (or local Postgres via Docker)
- Static Files: Served by Django automatically

In production:

- Web Server: **Gunicorn** (WSGI HTTP Server)
- Database: Managed PostgreSQL (AWS RDS, Render Postgres)
- Static Files: **WhiteNoise** (serves static files efficiently through Gunicorn)

11.2 Your **render.yaml** Configuration

YOUR project uses Render for deployment. This file defines the infrastructure as code.

```
services:
  - type: web
    name: finance-tracker
    env: python
    buildCommand: "./build.sh" # Installs deps, runs migrations, collectstatic
    startCommand: "gunicorn finance_tracker.wsgi:application"
    envVars:
      - key: DATABASE_URL
        fromDatabase:
          name: finance-db
          property: connectionString
      - key: SECRET_KEY
        generateValue: true
      - key: DEBUG
        value: "False"

databases:
  - name: finance-db
    databaseName: finance
    user: finance_user
```

11.3 The **build.sh** script

During deployment, you must run commands in a specific order:

```
#!/usr/bin/env bash
# exit on error
set -o errexit

pip install -r requirements.txt
python manage.py collectstatic --no-input
python manage.py migrate
```

11.4 Security Checklist before Deployment

1. **DEBUG = False**: NEVER deploy with **DEBUG=True**. It leaks secrets and source code.
2. **SECRET_KEY**: Must be a strong, random, secret environment variable.
3. **ALLOWED_HOSTS**: Must contain your production domain name (e.g. `['finance-tracker.onrender.com']`).
4. **SECURE_SSL_REDIRECT = True**: Forces HTTPS.

5. `SESSION_COOKIE_SECURE = True`: Ensures session cookies are only sent over HTTPS.

6. `CSRF_COOKIE_SECURE = True`: Ensures CSRF cookies are only sent over HTTPS.

11.5 Media Files in Production

`WhiteNoise` handles *static* files, but it does NOT handle user uploads (*media* files) because Heroku/Render file systems are ephemeral (they reset on every deploy).

If you want users to upload receipts in production, you MUST use an object storage service like AWS S3.

django-storages setup:

```
pip install boto3 django-storages
```

settings.py:

```
if not DEBUG:
    DEFAULT_FILE_STORAGE = 'storages.backends.s3boto3.S3Boto3Storage'
    AWS_ACCESS_KEY_ID = config('AWS_ACCESS_KEY_ID')
    AWS_SECRET_ACCESS_KEY = config('AWS_SECRET_ACCESS_KEY')
    AWS_STORAGE_BUCKET_NAME = config('AWS_STORAGE_BUCKET_NAME')
    AWS_S3_REGION_NAME = 'us-east-1'
```

CHAPTER 12 — INTERVIEW PREPARATION

If you are asked about Django in an interview based on your Finance Tracker project, use these structured answers.

A) MVT Architecture

- **30s:** "Django uses MVT: Models (DB schema), Views (request/business logic), Templates (presentation). URL routes call Views, Views use Models and feed Templates to render HTML."
- **2m:** "In my Finance Tracker, the flow goes: URL -> view -> view queries `Transaction.objects.filter(user=request.user)`. The view is thin; business logic belongs in my `services.py` layer. Templates use context variables for presentation. For APIs with DRF, the view returns serialized JSON rather than HTML."

B) Full request lifecycle

- **30s:** "Browser -> DNS/TCP/TLS -> web server -> WSGI/ASGI -> Django middleware -> URL resolver -> view -> ORM/services -> template/serializer -> HttpResponse -> middleware response -> client."
- **2m:** "Example: a request for `/transactions/` triggers `AuthenticationMiddleware` (session loaded). `URLConf` finds `transaction_list`, view executes and performs QuerySet (SQL sent to DB), then render."

Response middleware adds security headers before returning. For performance debugging, I capture which layer is slow and use Django debug toolbar."

C) ORM / N+1 problem

- **30s:** "N+1 happens when you query a list (1 query) and then access a relation per item (N extra queries). Fix with `select_related` for ForeignKeys or `prefetch_related` for ManyToMany."
- **2m:** "Example: rendering transactions and showing the category name. A naive loop causes N+1 queries. The fix: `Transaction.objects.filter(...).select_related('category')` makes one `LEFT JOIN` query. For tags, use `prefetch_related('tags')`, which issues 1 extra query with `IN (...)` and maps them in Python."

D) Migrations

- **30s:** "`makemigrations` inspects model state and generates migration files; `migrate` applies them to DB."
- **2m:** "Migrations are operations lists (`CreateModel`, `AddField`). For non-blocking production deploys, adopt the expand/backfill/contract pattern. I use `sqlmigrate` to preview generated SQL and watch for table rewrites and locking before deploying."

E) Session vs JWT auth

- **30s:** "Session auth uses server-side sessions + cookie (great for browser apps). JWT uses stateless signed tokens (common for mobile/API clients)."
- **2m:** "Session-based works well for my finance-tracker server-rendered UI: `login()` creates a server-side session stored in the DB; CSRF middleware protects POSTs. For an API consumed by mobile apps, JWT simplifies stateless auth: client stores token and sends an Authorization header. JWT requires a revocation strategy, though."

F) Middleware

- **30s:** "Middleware are hooks executed for every request/response to handle cross-cutting concerns (auth, sessions, CSRF, logging). Request-phase executes top-down, response-phase bottom-up."
- **2m:** "Order is crucial: `SessionMiddleware` must precede `AuthenticationMiddleware`. I use middleware for global behavior. For per-view behaviors, I prefer decorators to avoid global performance impact."

G) Django vs DRF: when to use which

- **30s:** "Use Django templating for server-rendered pages and admin; use DRF for API-first services or when mobile/third-party clients need JSON endpoints."
- **2m:** "My finance tracker uses server-rendered views for the user dashboard because sessions and CSRF are straightforward. For future mobile integrations, I add DRF endpoints. DRF provides serializers, browsable API, and consistent error formats. By sharing the `services.py` layer, I avoid duplicating business logic."

CHAPTER 13 — TROUBLESHOOTING CHEATSHEET

When things go wrong, consult this table.

Error Message / Symptom	Likely Cause	Fix
<code>ImportError</code> on startup	Bad <code>settings.py</code> or missing pip package	Check <code>DJANGO_SETTINGS_MODULE</code> and run <code>pip install -r requirements.txt</code> .
<code>OperationalError: no such column</code>	Model changed but DB didn't	Run <code>python manage.py makemigrations</code> then <code>python manage.py migrate</code> .
<code>TemplateDoesNotExist</code>	Django can't find HTML file	Ensure app is in <code>INSTALLED_APPS</code> and path exactly matches <code>templates/app_name/file.html</code> .
Forms saving but data missing	Forgot <code>commit=False</code> logic	If adding <code>request.user</code> to a form, do <code>obj = form.save(commit=False); obj.user = request.user; obj.save()</code> .
<code>IntegrityError: NOT NULL constraint failed</code>	Missing required field	Check model definition; ensure form or view provides all required fields (like <code>user</code>).
CSRF verification failed (403)	Missing token in POST form	Add <code>{% csrf_token %}</code> inside your HTML <code><form></code> .
Static files not loading in prod (404)	Missing <code>collectstatic</code>	Run <code>python manage.py collectstatic --no-input</code> on the server before starting Gunicorn.
DB changes missing on production	Uncommitted migrations	Always commit <code>0001_initial.py</code> etc. to Git so production can run <code>migrate</code> .
App changes not reflecting	<code>runserver</code> stale	Stop the server (Ctrl+C) and restart. Check if syntax errors killed the autoreloader.
Memory leaks / Slow performance	Heavy queries / N+1 problem	Use <code>django-debug-toolbar</code> to spot repeated queries. Add <code>select_related</code> .

End of Django Mastery Guide.

CHAPTER 14 — KNOWLEDGE CHECK & QUIZZES

This section contains the predict-then-reveal quizzes from your original notes. Test yourself to ensure you've mastered the concepts!

14.1 Templates, Static, & Media Quiz

Q1: If you add a file `assets/img/logo.png` and reference it via `{% static 'img/logo.png' %}`, what command(s) must you run / deploy steps must you take so the image is served in production? *Try to predict before reading below.*

Reveal A1:

- **Locally:** no command required for `DEBUG=True` dev server (dev static finders serve it).
- **Production:** you must run `python manage.py collectstatic` (this will copy `assets/img/logo.png` into `STATIC_ROOT/img/logo.png` or create a hashed filename if using manifest storage).
- Deploy static files (ensure `STATIC_ROOT` is accessible by Nginx or WhiteNoise) and confirm Nginx/WhiteNoise is configured to serve `STATIC_URL`.
- Also ensure `STATICFILES_DIRS` includes `assets/` (so `collectstatic` finds the file).

Q2: A user uploads a receipt named `receipt.jpg`. Given `MEDIA_ROOT=/srv/myproject/media` and `upload_to='receipts/%Y/%m/%d/'`, what will be the stored DB value and the absolute filesystem path for a 2026-06-23 upload?

Reveal A2:

- **DB value stored:** `'receipts/2026/06/23/receipt.jpg'`
- **Absolute filesystem path:** `/srv/myproject/media/receipts/2026/06/23/receipt.jpg` (assuming `MEDIA_ROOT = /srv/myproject/media`)

14.2 Authentication Quiz

Q1: After `login(request, user)` is called, where is the user identity stored on the server and what cookie does the browser receive?

Reveal A1:

- **Server-side:** session data stored according to `SESSION_ENGINE` — commonly in DB table `django_session` where `session_key` maps to a serialized session containing user id and backend path.
- **Browser:** receives `Set-Cookie: sessionid=<session_key>`; subsequent requests include `Cookie: sessionid=<session_key>`.

Q2: For an API consumed by a mobile app, why might session-based auth be awkward and JWT be preferable?

Reveal A2:

- Session-based auth requires browser cookies and CSRF protection, and server-side session storage. Mobile apps must manage cookies and cannot easily rely on browser CSRF flow.
- JWT gives a bearer token that the mobile client can store and include in Authorization headers; it's stateless (no server session lookup) and naturally fits token-based API auth. But it introduces token revocation and security complexity.

14.3 Middleware Quiz

Q1: If you put `AuthenticationMiddleware` before `SessionMiddleware` in the `MIDDLEWARE` list, what breaks?

Reveal A1:

- `AuthenticationMiddleware` will fail to find `request.session` (it expects `SessionMiddleware` to have already attached `request.session`), so `request.user` will not be properly set. This breaks login/session-based auth and decorators relying on `request.user`.

Q2: You want to add a middleware that forbids requests lacking a custom header `X-Client-Id`. Where should that middleware go (before/after `AuthenticationMiddleware`) and why?

Reveal A2:

- Put the header-checking middleware **before** `AuthenticationMiddleware` if you want to reject unauthenticated requests early (and avoid unnecessary session/user lookups).
- If the header is required for authentication itself, put it before or as part of an authentication middleware.
- If the header is just an additional check after auth (depends on `request.user`), put it **after** `AuthenticationMiddleware`.

CHAPTER 15 — RESTORED ORIGINAL CONTENT

Managers vs QuerySets

MANAGERS vs QUERYSETS (code-first)

Transactions / Atomic

transactions/atomic, and migrations internals. Each topic follows the pattern you requested: real minimal syntax first, then SQL shown, then why/when, performance note, and a common beginner mistake.

Note: when I show SQL, I use PostgreSQL style examples (psycopg2) since your project likely uses Postgres in production. For your local SQLite behavior the SQL differs in dialect but the conceptual mapping is the same.

DENSE REVISION CHEATSHEET

(Skimmable one-page style — keep this open as your memory aid)

A) Essential commands

- `python -m venv .venv`
- `source .venv/bin/activate`
- `pip install -r requirements.txt`
- `django-admin startproject projectname .`
- `python manage.py startapp appname`
- `python manage.py runserver`
- `python manage.py makemigrations`
- `python manage.py migrate`

- `python manage.py sqlmigrate app_name 000X`
- `python manage.py showmigrations`
- `python manage.py createsuperuser`
- `python manage.py shell`
- `python manage.py collectstatic --noinput`
- `python manage.py test`
- `python manage.py dumpdata app.Model > data.json`
- `python manage.py loaddata data.json`

B) Request flow diagram (compact)

```
Client -> DNS -> TCP/TLS -> Nginx (reverse proxy) -> Gunicorn/Uvicorn -> WSGI/ASGI
app
-> Django startup (settings, app registry) already happened
-> Middleware (request: top->down)
-> URL resolver -> view (FBV/CBV/APIView/ViewSet)
-> Business logic / service layer -> ORM queries -> DB
-> Serializer or Template render -> HttpResponse
-> Middleware (response: bottom->up)
-> Server -> Client
```

C) One-line purpose of each common file

- `manage.py`: per-project CLI entrypoint (sets `DJANGO_SETTINGS_MODULE`).
- `settings.py`: global configuration (apps, DB, middleware, static/media).
- `urls.py` (project): root URL router; include app urls.
- `wsgi.py` / `asgi.py`: production entrypoints for WSGI/ASGI servers.
- `apps.py`: `AppConfig` + `ready()` hook.
- `models.py`: ORM model classes -> DB schema.
- `views.py`: request handlers (thin orchestration).
- `templates/`: DTL files for HTML.
- `static/`: developer-owned assets before `collectstatic`.
- `forms.py`: form validation / `ModelForm`.
- `admin.py`: admin site registration/customization.
- `middleware.py`: global request/response hooks.
- `migrations/`: migration files (schema/data operations).

D) ORM quick-reference (method → what it does → SQL effect)

- `Model.objects.filter(**conds)` → `QuerySet` (`SELECT ... WHERE ...`) — no execute until evaluated.
- `.get(**conds)` → fetch single row (`LIMIT 1`) → raises `DoesNotExist`/`MultipleObjectsReturned`.
- `.exclude(...)` → `WHERE NOT ...`

- `.order_by('field')` → ORDER BY ...
- `.annotate(total=Sum('amount'))` → SELECT ... GROUP BY ... , SUM(amount)
- `.aggregate(Sum('amount'))` → SELECT SUM(amount) ...
- `.select_related('fk')` → add JOIN to SELECT (no extra queries)
- `.prefetch_related('m2m')` → extra query to fetch related rows then map in Python
- `.values('field')` → SELECT specific columns, returns dicts
- `.values_list('id', flat=True)` → SELECT column, return list of values
- `.only('a','b')` / `.defer('bigfield')` → load subset of columns
- `.update(...)` → single SQL UPDATE ... WHERE ... (no save hooks)
- `.delete()` → SQL DELETE (cascades per DB constraints)
- `.bulk_create([...])` → bulk INSERT, bypasses save()

E) Auth flow diagram (session-based)

```
User posts credentials -> authenticate() verifies credentials ->
login(request,user)
-> server stores session (django_session row) -> server returns Set-Cookie:
sessionid=<key>
-> subsequent request includes Cookie -> SessionMiddleware loads session ->
AuthenticationMiddleware sets request.user
```

F) Static vs Media (quick table)

- Static:
- Owner: developer
- Settings: `STATIC_URL`, `STATICFILES_DIRS`, `STATIC_ROOT`
- Deployment: `collectstatic` -> `STATIC_ROOT` -> served by Nginx/WhiteNoise/CDN
- Media:
- Owner: users (uploads)
- Settings: `MEDIA_URL`, `MEDIA_ROOT`, `DEFAULT_FILE_STORAGE`
- Deployment: served by Nginx or cloud storage (S3), require access controls as needed

G) DRF building blocks (one-line table)

- Serializer / ModelSerializer: defines JSON shape and validation (like Form for APIs).
- APIView: low-level class mapping HTTP verbs to methods; manual control.
- GenericAPIView + mixins: get implementations for list/create/retrieve/update/destroy.
- ViewSet / ModelViewSet: groups related actions and works with routers to generate URLs.
- Routers: auto-create RESTful route patterns (list/retrieve/etc).
- Authentication classes: `SessionAuthentication`, `TokenAuthentication`, `JWT`, etc.
- Permission classes: `IsAuthenticated`, `IsAdminUser`, custom `BasePermission`

- Throttling: UserRateThrottle, AnonRateThrottle; config in settings.
- Pagination: PageNumberPagination / LimitOffsetPagination; set in REST_FRAMEWORK settings.
- Parsers: JSONParser, FormParser, MultiPartParser (handle file uploads).
- Renderers: JSONRenderer,BrowsableAPIRenderer (HTML), XMLRenderer.

H) Quick DRF code snippets

- Register ViewSet with router (urls.py):

```
from rest_framework.routers import DefaultRouter
router = DefaultRouter()
router.register('transactions', TransactionViewSet)
urlpatterns = [
    path('api/', include(router.urls)),
    ## ]
```

- Protect endpoint:

```
class TransactionViewSet(viewsets.ModelViewSet):
    permission_classes = [IsAuthenticated]
```

I) What to say in interviews (1-line cheat)

- If asked "where to add a new endpoint?": add Serializer + ViewSet (or APIView) + register in router -> reuse service layer -> secure with permissions and throttling -> add tests.
- If asked "how to fix slow page": identify slow layer (SQL vs template vs external API). For SQL use select_related/prefetch, add indexes, use EXPLAIN ANALYZE; for template heavy-lifting, cache fragments; for external calls, background tasks.
- If asked "how to migrate schema safely": expand/backfill/contract; use RunPython to backfill; prefer concurrent index creation on Postgres.

CHAPTER 16 — ORIGINAL EXAMPLES & ADVANCED DETAILS

Example 3 — Add a new Budget category with a monthly limit

Interpretation: Add a Budget model that links user + category + monthly_limit; implement model, migration, admin, form, view, and template to create a budget for a category.

Files changed/created:

- finance/models.py — add Budget model
- finance/migrations/000X_create_budget.py — migration created by makemigrations
- finance/admin.py — register Budget

- finance/forms.py — BudgetForm
- finance/views.py — add budget_create_view and budget_list_view
- finance/urls.py — add routes
- templates/finance/budget_form.html, budget_list.html — templates
- tests — add tests for budget creation and uniqueness

Step-by-step code

1. models.py — Budget model

```
# finance/models.py
from django.conf import settings
from django.db import models
from decimal import Decimal

class Budget(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL,
                             on_delete=models.CASCADE, related_name='budgets')
    category = models.ForeignKey('Category', on_delete=models.PROTECT,
                                 related_name='budgets')
    monthly_limit = models.DecimalField(max_digits=12, decimal_places=2)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ('user', 'category') # one budget per user per category
        indexes = [
            models.Index(fields=['user', 'category']),
        ]

    def __str__(self):
        return f'{self.user.username} - {self.category.name} limit {self.monthly_limit}'
```

Why unique_together: prevent duplicate budgets for same user/category.

2. admin.py — register Budget

```
# finance/admin.py
from .models import Budget

@admin.register(Budget)
class BudgetAdmin(admin.ModelAdmin):
    list_display = ('user', 'category', 'monthly_limit')
    list_filter = ('category',)
    search_fields = ('user__username',)
```

3. forms.py — BudgetForm

```
# finance/forms.py

from django import forms
from .models import Budget

class BudgetForm(forms.ModelForm):
    class Meta:
        model = Budget
        fields = ['category', 'monthly_limit']
```

4. views.py — create/list budgets

```
# finance/views.py
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, redirect, get_object_or_404
from .forms import BudgetForm
from .models import Budget

## @login_required
def budget_list(request):
    budgets = Budget.objects.filter(user=request.user).select_related('category')
    return render(request, 'finance/budget_list.html', {'budgets': budgets})

## @login_required
def budget_create(request):
    if request.method == 'POST':
        form = BudgetForm(request.POST)
        if form.is_valid():
            # create with user
            budget = form.save(commit=False)
            budget.user = request.user
            budget.save()
            return redirect('finance:budget_list')
        else:
            form = BudgetForm()
            return render(request, 'finance/budget_form.html', {'form': form})
```

5. urls.py — add routes

```
# finance/urls.py
urlpatterns += [
    path('budgets/', views.budget_list, name='budget_list'),
    path('budgets/create/', views.budget_create, name='budget_create'),
    ## ]
```

6. templates — budget_form.html

```
{# templates/finance/budget_form.html #}
{% extends "base.html" %}
{% block content %}
<h1>Create Budget</h1>
<form method="post">
{% csrf_token %}
{{ form.as_p }}
<button type="submit">Create</button>
## </form>
{% endblock %}
```

7. templates — budget_list.html

```
{# templates/finance/budget_list.html #}
{% extends "base.html" %}
{% block content %}
<h1>Your Budgets</h1>
## <ul>
{% for b in budgets %}
<li>{{ b.category.name }} - {{ b.monthly_limit }}</li>
{% empty %}
<li>No budgets yet</li>
{% endfor %}
## </ul>
<a href="{% url 'finance:budget_create' %}">Create Budget</a>
{% endblock %}
```

8. migrations

Commands:

```
python manage.py makemigrations finance
python manage.py migrate
```

Example migration file (auto-generated):

```
class Migration(migrations.Migration):
    dependencies = [
        ('finance', '000X_previous'),
        ## ]
    operations = [
        migrations.CreateModel(
            name='Budget',
            fields=[
                ('id', models.AutoField(primary_key=True, ...)),
```

```

('monthly_limit', models.DecimalField(max_digits=12, decimal_places=2)),
('created_at', models.DateTimeField(auto_now_add=True)),

('user', models.ForeignKey(on_delete=..., to=settings.AUTH_USER_MODEL)),
('category', models.ForeignKey(on_delete=..., to='finance.Category')),
## ],
options={'unique_together': (('user', 'category'))},
## ),
migrations.AddIndex(
model_name='budget',
index=models.Index(fields=['user', 'category'],
name='finance_budget_user_category_idx'),
## ),
## ]

```

SQL preview (sqlmigrate):

```

CREATE TABLE finance_budget (
id serial PRIMARY KEY,
monthly_limit numeric(12,2) NOT NULL,
created_at timestamp with time zone NOT NULL,
user_id integer NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
category_id integer NOT NULL REFERENCES finance_category(id) ON DELETE
## CASCADE
## );
CREATE UNIQUE INDEX finance_budget_user_category ON finance_budget (user_id,
category_id);
CREATE INDEX finance_budget_user_category_idx ON finance_budget (user_id,
category_id);

```

9. tests — create budget test

```

# finance/tests.py
class BudgetTests(TestCase):
def setUp(self):
self.user = User.objects.create_user(username='b', password='p')
self.client = Client()
self.client.login(username='b', password='p')
self.category = Category.objects.create(name='Health')

def test_create_budget(self):
resp = self.client.post('/finance/budgets/create/', {'category': self.category.id,
'monthly_limit': '150.00'}, follow=True)
self.assertRedirects(resp, '/finance/budgets/')
self.assertTrue(Budget.objects.filter(user=self.user, category=self.category,
monthly_limit='150.00').exists())

def test_unique_budget_per_user_category(self):
Budget.objects.create(user=self.user, category=self.category, monthly_limit='100')

```

```
resp = self.client.post('/finance/budgets/create/', {'category': self.category.id,
## 'monthly_limit': '200'})
self.assertContains(resp, 'Budget with this User and Category already exists',
status_code=200)
```

Note: the default model `unique_together` will raise `IntegrityError` at DB level; better to add form validation to catch and show user-friendly errors. Add form `clean` to check existing budget.

10. form validation for uniqueness (optional)

```
class BudgetForm(forms.ModelForm):
class Meta:
model = Budget
fields = ['category', 'monthly_limit']

def __init__(self, *args, **kwargs):
self.user = kwargs.pop('user', None)
super().__init__(*args, **kwargs)

def clean(self):
cleaned = super().clean()
category = cleaned.get('category')
if self.user and Budget.objects.filter(user=self.user,
category=category).exists():
raise forms.ValidationError("You already have a budget for this category.")
return cleaned
```

And adjust view to pass user to form on POST: `form = BudgetForm(request.POST, user=request.user)`

11. run & verify

Commands:

```
python manage.py makemigrations finance
python manage.py migrate
python manage.py runserver
python manage.py test finance.tests.BudgetTests
```

Notes and cautions

- DB uniqueness is final guard; always validate on form level to present friendly messages and to avoid `IntegrityError` on save. If saving without form validation, wrap in `try/except` to catch `IntegrityError` and show message.
- Consider currency precision and rounding rules for `monthly_limit` (`DecimalField` with appropriate `max_digits/decimal_places`). Use `Decimal` in Python.

Example 3 complete.

Final notes for all three examples

- Always run `python manage.py makemigrations` and inspect generated migration before applying in production. Use `python manage.py sqlmigrate appname migration_number` to preview SQL and assess lock/impact.
- Write tests for model behavior (validation), view behavior (permission/redirects), and integration (file storage).
- For file uploads in tests, use `SimpleUploadedFile` and make sure `MEDIA_ROOT` during tests points to temp dir; Django's `TestCase` isolates DB but not media storage — you may need to override `settings.MEDIA_ROOT` in tests to a temp dir and cleanup.
- Add new apps (if you used `startapp`) to `INSTALLED_APPS` then run migrations.
- When changing templates, ensure static references use `{% load static %}` and `{% static 'path' %}`.

What you should be able to do now

- Implement a OneToOne Profile with `ImageField`, wire up forms/views/templates, and test upload handling.
- Add GET-based search/filtering to list views with safe ORM usage (`select_related/prefetch_related`) and paginate results.
- Add a new Budget model, ensure uniqueness, create forms/views/templates, and write tests covering DB uniqueness and form validation.

SECTION C — LAZY EVALUATION (proof & code)

Example code:

```
qs = Transaction.objects.filter(user=user)    # NO SQL yet
print(type(qs))    # QuerySet
# SQL executed at iteration:
for tx in qs[:10]:
    print(tx.id)    # triggers SQL
```

Proof (inspect SQL without executing):

```
qs = Transaction.objects.filter(user=user, amount__lt=0)
print(str(qs.query))
# or in Django 3.2+: print(qs.query.__str__())
```

The printed SQL:

```
SELECT "finance_transaction"."id", "finance_transaction"."amount", ...
FROM "finance_transaction"
WHERE "finance_transaction"."user_id" = 5 AND "finance_transaction"."amount" < 0
ORDER BY "finance_transaction"."date" DESC;
```

Demonstration of laziness: constructing `qs` does not touch DB; methods like `.count()`, `list(qs)`, `.exists()`, iterating, slicing (that evaluates) will run SQL.

Why/When: lazy behavior allows composing queries efficiently.

Performance note: calling `len(qs)` forces evaluation — use `qs.count()` when you need DB count (but `count()` executes SQL COUNT which may be faster than fetching rows).

Common mistake: calling `list(qs)` in template or code unnecessarily causing large memory usage.

3) Session-based auth vs JWT — code/config differences and when to use each

Session-based (Django default)

- Uses server-side session storage and cookie (sessionid) to identify user.
- Code: `authenticate()`, `login()`, `request.user` via `AuthenticationMiddleware`.
- Pros:
- Simple to implement (built-in).
- Server controls session lifetime and can revoke sessions (delete session on server).
- CSRF protection works naturally for browser forms.
- Cons:
- Not ideal for public APIs consumed by third-party clients (mobile apps) unless you implement token adapters.
- Requires session store scaling (DB/Cache/redis).

JWT (JSON Web Token) example (DRF + simplejwt)

- Setup (pip install `django-rest-framework-simplejwt`), urls:

```
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView

urlpatterns += [
    path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
    ## ]
```

Client obtains access and refresh tokens, then includes `Authorization: Bearer <access_token>` header in subsequent API requests. Backend validates token signature and expiry; typically no server-side session lookup.

Differences in practice

- Session: cookie-based; CSRF required for unsafe HTTP methods from browsers; better for server-rendered pages.
- JWT: stateless (unless you keep a denylist/refresh store); useful for mobile/third-party clients, microservices, and when you want token-based auth in APIs.

When to use which

- Use session-based auth for server-rendered sites (regular Django templates) where users authenticate via browser.
- Use JWT for API-first services consumed by mobile/native apps, or when you need stateless auth across multiple services. Consider security tradeoffs: token revocation, refresh handling, token theft risk.

Part 1 — QUIZ (predict-then-reveal)

Q1: If you add a file `assets/img/logo.png` and reference it via `{% static 'img/logo.png' %}`, what command(s) must you run/deploy steps must you take so the image is served in production? Try to predict (files changed and commands). Answer below.

Q2: A user uploads a receipt named `receipt.jpg`. Given `MEDIA_ROOT=/srv/myproject/media` and `upload_to='receipts/%Y/%m/%d/'`, what will be the stored DB value and the absolute filesystem path for a 2026-06-23 upload? Predict, then reveal below.

Reveal answers — check after you predict:

A1:

- Locally: no command required for `DEBUG=True` dev server (dev static finders serve it), but in production you must run:
- `python manage.py collectstatic` (this will copy `assets/img/logo.png` into `STATIC_ROOT/img/logo.png` or create a hashed filename if using manifest storage).
- Deploy static files (ensure `STATIC_ROOT` is accessible by Nginx or WhiteNoise) and confirm Nginx/WhiteNoise configured to serve `STATIC_URL`.
- Also ensure `STATICFILES_DIRS` includes `assets/` (so `collectstatic` finds the file).

A2:

- DB value stored: `'receipts/2026/06/23/receipt.jpg'`
- Absolute filesystem path: `/srv/myproject/media/receipts/2026/06/23/receipt.jpg` (assuming `MEDIA_ROOT = /srv/myproject/media`)

Part 2 — QUIZ (predict-then-reveal)

Q1: After `login(request, user)` is called, where is the user identity stored on the server and what cookie does the browser receive? Predict and then reveal below.

Q2: For an API consumed by a mobile app, why might session-based auth be awkward and JWT be preferable? Predict then reveal.

Reveal answers — check after you predict:

A1:

- Server-side: session data stored according to SESSION_ENGINE — commonly in DB table `django_session` where `session_key` maps to a serialized session containing user id and backend path.
- Browser receives Set-Cookie: `sessionid=<session_key>`; subsequent requests include Cookie: `sessionid=<session_key>`.

A2:

- Session-based auth requires browser cookies and CSRF protection, and server-side session storage; mobile apps must manage cookies and cannot easily rely on browser CSRF flow. JWT gives a bearer token that the mobile client can store and include in Authorization headers; it's stateless (no server session lookup) and naturally fits token-based API auth. But it introduces token revocation and security complexity.

Part 3 — QUIZ (predict-then-reveal)

Q1: If you put `AuthenticationMiddleware` before `SessionMiddleware` in the `MIDDLEWARE` list, what breaks? Predict then reveal.

Q2: You want to add a middleware that forbids requests lacking a custom header `X-Client-Id`. Where should that middleware go (before/after `AuthenticationMiddleware`) and why? Predict then reveal.

Reveal answers — check after you predict:

A1:

- `AuthenticationMiddleware` will fail to find `request.session` (it expects `SessionMiddleware` to have already attached `request.session`), so `request.user` will not be properly set (or it may behave unpredictably). This breaks login/session-based auth and decorators relying on `request.user`.

A2:

- Put the header-checking middleware before `AuthenticationMiddleware` if you want to reject unauthenticated requests early (and avoid unnecessary session/user lookups). If header is required for authentication itself (e.g., header contains token used to authenticate and you need that to set `request.user`), put it before or as part of an authentication middleware. If the header is just an additional check after auth, put it after `AuthenticationMiddleware`. In

summary: place it where its inputs are available—before `AuthenticationMiddleware` to short-circuit and reject early, or after if it depends on `request.user`.

— End of Prompt 3 output.

CHAPTER 15 — RESTORED ORIGINAL CONTENT

Managers vs QuerySets

MANAGERS vs QUERYSETS (code-first)

Transactions / Atomic

transactions/atomic, and migrations internals. Each topic follows the pattern you requested: real minimal syntax first, then SQL shown, then why/when, performance note, and a common beginner mistake.

Note: when I show SQL, I use PostgreSQL style examples (psycopg2) since your project likely uses Postgres in production. For your local SQLite behavior the SQL differs in dialect but the conceptual mapping is the same.

DENSE REVISION CHEATSHEET

(Skimmable one-page style — keep this open as your memory aid)

A) Essential commands

- `python -m venv .venv`
- `source .venv/bin/activate`
- `pip install -r requirements.txt`
- `django-admin startproject projectname .`
- `python manage.py startapp appname`
- `python manage.py runserver`
- `python manage.py makemigrations`
- `python manage.py migrate`
- `python manage.py sqlmigrate app_name 000X`
- `python manage.py showmigrations`
- `python manage.py createsuperuser`
- `python manage.py shell`
- `python manage.py collectstatic --noinput`
- `python manage.py test`
- `python manage.py dumpdata app.Model > data.json`
- `python manage.py loaddata data.json`

B) Request flow diagram (compact)

```
Client -> DNS -> TCP/TLS -> Nginx (reverse proxy) -> Gunicorn/Uvicorn -> WSGI/ASGI app
```

```
-> Django startup (settings, app registry) already happened
-> Middleware (request: top->down)
-> URL resolver -> view (FBV/CBV/APIView/ViewSet)
-> Business logic / service layer -> ORM queries -> DB
-> Serializer or Template render -> HttpResponse
-> Middleware (response: bottom->up)
-> Server -> Client
```

C) One-line purpose of each common file

- `manage.py`: per-project CLI entrypoint (sets `DJANGO_SETTINGS_MODULE`).
- `settings.py`: global configuration (apps, DB, middleware, static/media).
- `urls.py` (project): root URL router; include app urls.
- `wsgi.py` / `asgi.py`: production entrypoints for WSGI/ASGI servers.
- `apps.py`: `AppConfig` + `ready()` hook.
- `models.py`: ORM model classes -> DB schema.
- `views.py`: request handlers (thin orchestration).
- `templates/`: DTL files for HTML.
- `static/`: developer-owned assets before `collectstatic`.
- `forms.py`: form validation / `ModelForm`.
- `admin.py`: admin site registration/customization.
- `middleware.py`: global request/response hooks.
- `migrations/`: migration files (schema/data operations).

D) ORM quick-reference (method → what it does → SQL effect)

- `Model.objects.filter(**conds)` → `QuerySet` (`SELECT ... WHERE ...`) — no execute until evaluated.
- `.get(**conds)` → fetch single row (`LIMIT 1`) → raises `DoesNotExist`/`MultipleObjectsReturned`.
- `.exclude(...)` → `WHERE NOT ...`
- `.order_by('field')` → `ORDER BY ...`
- `.annotate(total=Sum('amount'))` → `SELECT ... GROUP BY ... , SUM(amount)`
- `.aggregate(Sum('amount'))` → `SELECT SUM(amount) ...`
- `.select_related('fk')` → add `JOIN` to `SELECT` (no extra queries)
- `.prefetch_related('m2m')` → extra query to fetch related rows then map in Python
- `.values('field')` → `SELECT` specific columns, returns dicts
- `.values_list('id', flat=True)` → `SELECT` column, return list of values
- `.only('a','b') / .defer('bigfield')` → load subset of columns
- `.update(...)` → single SQL `UPDATE ... WHERE ...` (no save hooks)
- `.delete()` → SQL `DELETE` (cascades per DB constraints)

- `.bulk_create([...])` → bulk INSERT, bypasses `save()`

E) Auth flow diagram (session-based)

```
User posts credentials -> authenticate() verifies credentials ->
login(request,user)
-> server stores session (django_session row) -> server returns Set-Cookie:
sessionid=<key>
-> subsequent request includes Cookie -> SessionMiddleware loads session ->
AuthenticationMiddleware sets request.user
```

F) Static vs Media (quick table)

- Static:
- Owner: developer
- Settings: `STATIC_URL`, `STATICFILES_DIRS`, `STATIC_ROOT`
- Deployment: `collectstatic` -> `STATIC_ROOT` -> served by Nginx/WhiteNoise/CDN
- Media:
- Owner: users (uploads)
- Settings: `MEDIA_URL`, `MEDIA_ROOT`, `DEFAULT_FILE_STORAGE`
- Deployment: served by Nginx or cloud storage (S3), require access controls as needed

G) DRF building blocks (one-line table)

- Serializer / ModelSerializer: defines JSON shape and validation (like Form for APIs).
- APIView: low-level class mapping HTTP verbs to methods; manual control.
- GenericAPIView + mixins: get implementations for list/create/retrieve/update/destroy.
- ViewSet / ModelViewSet: groups related actions and works with routers to generate URLs.
- Routers: auto-create RESTful route patterns (list/retrieve/etc).
- Authentication classes: `SessionAuthentication`, `TokenAuthentication`, JWT, etc.
- Permission classes: `IsAuthenticated`, `IsAdminUser`, custom `BasePermission`
- Throttling: `UserRateThrottle`, `AnonRateThrottle`; config in settings.
- Pagination: `PageNumberPagination` / `LimitOffsetPagination`; set in `REST_FRAMEWORK` settings.
- Parsers: `JSONParser`, `FormParser`, `MultiPartParser` (handle file uploads).
- Renderers: `JSONRenderer`, `BrowsableAPIRenderer` (HTML), `XMLRenderer`.

H) Quick DRF code snippets

- Register ViewSet with router (`urls.py`):

```
from rest_framework.routers import DefaultRouter
router = DefaultRouter()
router.register('transactions', TransactionViewSet)
urlpatterns = [
    path('api/', include(router.urls)),
    ## ]
```

- Protect endpoint:

```
class TransactionViewSet(viewsets.ModelViewSet):
    permission_classes = [IsAuthenticated]
```

I) What to say in interviews (1-line cheat)

- If asked "where to add a new endpoint?": add Serializer + ViewSet (or APIView) + register in router -> reuse service layer -> secure with permissions and throttling -> add tests.
- If asked "how to fix slow page": identify slow layer (SQL vs template vs external API). For SQL use select_related/prefetch, add indexes, use EXPLAIN ANALYZE; for template heavy-lifting, cache fragments; for external calls, background tasks.
- If asked "how to migrate schema safely": expand/backfill/contract; use RunPython to backfill; prefer concurrent index creation on Postgres.

CHAPTER 17 — FULFILLING IMPLEMENTATION PLAN GAPS (NEW CHECKLISTS & TEMPLATES)

This section completes the final missing promises from the Implementation Plan, mapping deeper into your project structure.

17.1 Advanced Template Deep Dive (`base.html` & Partials)

#`base.html` Breakdown Your project uses `base.html` as the master layout. All other templates inherit from it.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Finance Tracker{% endblock %}</title>
    <!-- 1. Load static CSS -->
    {% load static %}
    <link rel="stylesheet" href="{% static 'css/main.css' %}">
</head>
<body>
    <!-- 2. Template Partial: Include the Navbar -->
    {% include "includes/nav.html" %}

    <div class="container">
        <!-- 3. Messages Framework (Flash Messages) -->
        {% if messages %}
            <ul class="messages">
                {% for message in messages %}
                    <li class="{% message.tags %}">{{ message }}</li>
                {% endfor %}
            </ul>
        {% endif %}
    </div>
</body>
</html>
```

```

        <!-- 4. Content Block -->
        {% block content %}
        {% endblock %}
    </div>
</body>
</html>

```

What `{% include %}` does (Template Partials)

What: `{% include "includes/nav.html" %}` injects the exact HTML of `nav.html` into `base.html`. **Why:** Keeps `base.html` clean. You can reuse `nav.html` anywhere. The included template has access to the full context (so `{{ request.user }}` works inside `nav.html`).

17.2 Extending UserCreationForm

Django provides `UserCreationForm`, but it only asks for username/password. For the Finance Tracker, you likely want an email address too.

```

# core/forms.py
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth import get_user_model
from django import forms

User = get_user_model()

class UserRegistrationForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta(UserCreationForm.Meta):
        model = User
        # Add email to the list of fields
        fields = UserCreationForm.Meta.fields + ('email',)

```

Why: Extends the default form securely. It automatically hashes the password using `UserCreationForm`'s built-in `save()` method, but now enforces email collection.

17.3 The 5 Practical Checklists

Keep these checklists handy when you want to add new features to your project.

Checklist 1: "Add a new model field"

1. **Edit `models.py`:** Add the field. *Rule: If the table already has data, you MUST provide `null=True` or `default=...`*

```
currency = models.CharField(max_length=3, default='USD')
```

2. **Make migrations:** `python manage.py makemigrations`
3. **Inspect SQL:** `python manage.py sqlmigrate appname 000X` (Check if it causes table locks!)
4. **Migrate DB:** `python manage.py migrate`
5. **Update Forms:** Add the field to `fields = [...]` in your `ModelForm`.
6. **Update Templates:** Ensure the new field is rendered in HTML.
7. **Update Admin:** Add the field to `list_display` in `admin.py`.

Checklist 2: "Add a new page/route"

1. **Create the View (`views.py`):**

```
def about_view(request):  
    return render(request, 'about.html')
```

2. **Wire the URL (`urls.py`):**

```
path('about/', views.about_view, name='about'),
```

3. **Create the Template (`templates/about.html`):**

```
{% extends "base.html" %}  
{% block content %}<h1>About</h1>{% endblock %}
```

4. **Link to it in Nav:** `<a href="{% url 'about' %}">About</a>`

Checklist 3: "Add file upload to a model"

1. **Install Pillow:** `pip install Pillow` (only needed for `ImageField`).
2. **Model:** Add `receipt = models.FileField(upload_to='receipts/', blank=True)`.
3. **Migrate:** `makemigrations` and `migrate`.
4. **Form:** Ensure it is a `ModelForm` with `receipt` in the `fields` list.
5. **Template:** YOU MUST update your `<form>` tag: `<form method="POST" enctype="multipart/form-data">`.
6. **View:** Bind `request.FILES` to the form: `form = MyForm(request.POST, request.FILES)`.
7. **Settings:** Verify `MEDIA_URL` and `MEDIA_ROOT` are set.

Checklist 4: "Add a new Django app"

1. **Run Command:** `python manage.py startapp reports`
2. **Register App:** Add `'reports'`, to `INSTALLED_APPS` in `settings.py`.

3. **URL Routing:** In project `urls.py`, add `path('reports/', include('reports.urls'))`,.
4. **Create App URLs:** Create `reports/urls.py`.
5. **Write Models/Views:** Add your logic, then run `makemigrations reports` and `migrate`.

Checklist 5: "Add a new third-party package"

Example: adding `django-crispy-forms` to make forms look better.

1. **Install:** `pip install django-crispy-forms`
2. **Freeze:** `pip freeze > requirements.txt` (Crucial so production server knows about it).
3. **Register:** Add `'crispy_forms'` to `INSTALLED_APPS` in `settings.py`.
4. **Configure:** Follow the docs (e.g., `CRISPY_TEMPLATE_PACK = 'bootstrap4'`).
5. **Use in Template:** `{% load crispy_forms_tags %}` and `{{ form|crispy }}`.